

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN



ĐỒ ÁN MÔN HỌC
THIẾT KẾ HỆ THỐNG SỐ HDL
THIẾT KẾ VÀ THỰC THI BỘ MÃ HÓA VÀ GIẢI
MÃ ẢNH AES-128 CHO ẢNH TĨNH TRÊN FPGA

GVHD: ThS. Hồ Ngọc Diễm

LỚP: CE213.Q22

SINH VIÊN THỰC HIỆN	MÃ SỐ SINH VIÊN
Trần Duy Nhân	23521088
Nguyễn Hoàng Đăng Khoa	23520749

TP. HỒ CHÍ MINH – 2026

MỤC LỤC

MỤC LỤC	i
DANH MỤC VIẾT TẮT	iv
DANH MỤC HÌNH ẢNH	v
DANH MỤC BẢNG BIỂU	vii
LỜI MỞ ĐẦU	ix
Bảng Phân Công	x
CHƯƠNG 1. MỞ ĐẦU	1
1.1 Lý do chọn đề tài	1
1.2 Mục tiêu đề tài	2
1.3 Đối tượng và phạm vi thực hiện	3
1.4 Phương pháp thực hiện	4
CHƯƠNG 2. CƠ SỞ LÝ THUYẾT VÀ CÔNG NGHỆ LIÊN QUAN	5
2.1 Tổng quan về thuật toán AES-128.....	5
2.1.1 Khái niệm mã hóa khối và AES-128	6
2.1.2 Cấu trúc dữ liệu 128 bit trong AES	6
2.1.3 Tổng quan quá trình mã hóa AES-128	7
2.1.4 Tổng quan quá trình giải mã AES-128.....	8
2.2 Các phép biến đổi trong mã hóa AES-128	9
2.2.1 AddRoundKey	9
2.2.2 SubBytes và S-Box	10
2.2.3 ShiftRows.....	10
2.2.4 MixColumns	11
2.2.5 KeyExpansion, RotWord, SubWord và Rcon	11
2.3 Các phép biến đổi trong giải mã AES-128	12
2.4 Biểu diễn ảnh RGB565	13
2.5 Giao tiếp UART trên kit FPGA	15

2.6 Cơ chế packet, CRC8 và ACK/NACK.....	15
2.7 Bộ nhớ ngoài SRAM trên kit FPGA.....	17
2.8 DMA trong hệ thống xử lý ảnh trên FPGA	19
2.9 Chuẩn VGA 640x480 và hiển thị ảnh từ SRAM.....	19
CHƯƠNG 3. PHÂN TÍCH YÊU CẦU VÀ THIẾT KẾ HỆ THỐNG	22
3.1 Tổng quan bài toán	22
3.2 Yêu cầu chức năng và phi chức năng	22
3.3 Kiến trúc tổng quan của hệ thống	23
3.4 Thiết kế cấu trúc thiết kế và module.....	25
3.5 Thiết kế khối giao tiếp UART và nạp ảnh.....	27
3.6 Thiết kế tổ chức bộ nhớ SRAM.....	28
3.7 Thiết kế khối phân xử truy cập SRAM.....	30
3.8 Thiết kế DMA xử lý dữ liệu ảnh.....	31
3.9 Thiết kế AES-128 core wrapper	32
3.10 Thiết kế controller hệ thống.....	33
3.11 Thiết kế hiển thị VGA và dashboard debug	34
3.12 Thiết kế phần mềm gửi ảnh từ máy tính.....	35
3.13 Tổng kết thiết kế hệ thống	36
CHƯƠNG 4. THỰC HIỆN, KIỂM THỬ VÀ ĐÁNH GIÁ KẾT QUẢ.....	37
4.1 Môi trường thực hiện	37
4.2 Tổ chức mã nguồn thực hiện	37
4.3 Kiểm thử AES core wrapper.....	38
4.4 Kiểm thử UART packet và CRC8	39
4.5 Kiểm thử SRAM arbiter	40
4.6 Kiểm thử AES SRAM DMA	41
4.7 Kiểm thử VGA timing và hiển thị	42
4.8 Kiểm thử tích hợp toàn hệ thống	44
4.9 Kết quả tổng hợp trên Quartus.....	45
4.10 Phân tích kết quả demo ảnh	46

4.11 Hạn chế trong phạm vi kiểm thử	50
CHƯƠNG 5. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	52
5.1 Mức độ hoàn thành mục tiêu	52
5.2 Ưu điểm của hệ thống.....	52
5.3 Hạn chế còn tồn tại	53
5.4 Hướng phát triển.....	54
5.5 Kết luận chung.....	55
DANH MỤC TÀI LIỆU THAM KHẢO.....	56

DANH MỤC VIẾT TẮT**Bảng 1. Danh mục thuật ngữ viết tắt sử dụng trong báo cáo**

Viết tắt	Thuật ngữ đầy đủ	Ý nghĩa
AES	Advanced Encryption Standard	Chuẩn mã hóa đối xứng được sử dụng để mã hóa và giải mã dữ liệu theo từng block.
ACK	Acknowledgement	Tín hiệu phản hồi cho biết packet đã được nhận hợp lệ.
CRC	Cyclic Redundancy Check	Cơ chế kiểm tra lỗi dữ liệu trong quá trình truyền packet.
DMA	Direct Memory Access	Cơ chế đọc ghi dữ liệu khối giữa bộ nhớ và module xử lý mà không cần CPU.
FPGA	Field-Programmable Gate Array	Mạch tích hợp có khả năng cấu hình lại bằng mô tả phần cứng.
FSM	Finite State Machine	Máy trạng thái hữu hạn dùng để điều khiển luồng hoạt động.
HDL	Hardware Description Language	Ngôn ngữ mô tả phần cứng, trong đề tài sử dụng Verilog HDL.
RGB565	Red Green Blue 5-6-5	Định dạng màu 16 bit gồm 5 bit đỏ, 6 bit xanh lá và 5 bit xanh dương.
RTL	Register Transfer Level	Mức mô tả thiết kế phần cứng theo thanh ghi và tổ hợp logic.
SRAM	Static Random Access Memory	Bộ nhớ ngoài dùng để lưu ảnh gốc, ảnh mã hóa và ảnh giải mã.
UART	Universal Asynchronous Receiver/Transmitter	Chuẩn truyền nối tiếp bất đồng bộ dùng để nạp ảnh từ máy tính xuống FPGA.
VGA	Video Graphics Array	Chuẩn hiển thị ảnh ra màn hình thông qua tín hiệu đồng bộ và màu.

DANH MỤC HÌNH ẢNH

Hình 1. Tổng quan bài toán mã hóa và giải mã ảnh trên FPGA.....	1
Hình 2. Biểu diễn block 128 bit thành ma trận trạng thái AES	6
Hình 3. Cách ghép 8 pixel RGB565 thành một block AES 128 bit	7
Hình 4. Quá trình mã hóa AES-128.....	8
Hình 5. Quá trình giải mã AES-128	9
Hình 6. Minh họa phép SubBytes và bảng S-Box	10
Hình 7. Minh họa phép MixColumns trên một cột trạng thái	11
Hình 8. Quá trình KeyExpansion trong AES-128	12
Hình 9. Mối quan hệ giữa AES encryption và AES decryption.....	13
Hình 10. Cấu trúc định dạng RGB565.....	14
Hình 11. Cấu trúc khung truyền UART 8N1.....	15
Hình 12. Sơ đồ cấu trúc packet UART và cơ chế ACK/NACK.....	17
Hình 13. Sơ đồ phân vùng SRAM cho ảnh gốc, ảnh mã hóa và ảnh giải mã	18
Hình 14. Luồng hoạt động của DMA giữa SRAM và AES core	19
Hình 15. Bố cục VGA 640x480 chia bốn vùng.....	20
Hình 16. Luồng xử lý tổng quát của hệ thống đề xuất	22
Hình 17. Sơ đồ khối tổng thể của hệ thống đề xuất.....	24
Hình 18. Cấu trúc thư mục thiết kế hệ thống đề xuất.....	27
Hình 19. Sơ đồ luồng nạp ảnh qua UART vào SRAM	28
Hình 20. Address map SRAM của hệ thống.....	29
Hình 21. Sơ đồ phân xử SRAM giữa UART, DMA và VGA.....	30
Hình 22. FSM đề xuất của AES SRAM DMA.....	31
Hình 23. Giao diện chuẩn hóa của khối giao diện lõi AES	32
Hình 24. FSM điều khiển tổng thể của hệ thống	33
Hình 25. Kiến trúc VGA subsystem và dashboard.....	35
Hình 26. Luồng xử lý của phần mềm gửi ảnh trên PC	36
Hình 27. Waveform kiểm thử AES wrapper với known-answer vector	39
Hình 28. Waveform kiểm thử UART packet đúng CRC và sai CRC	40

Hình 29. Waveform kiểm thử ưu tiên SRAM arbiter	41
Hình 30. Waveform kiểm thử DMA đọc SRAM, xử lý AES và ghi kết quả	42
Hình 31. Waveform kiểm thử VGA timing 640x480	43
Hình 32. Ảnh chụp VGA sau khi nạp ảnh gốc	43
Hình 33. Ảnh chụp VGA sau khi mã hóa và giải mã	44
Hình 34. Waveform smoke test tích hợp hệ thống	45
Hình 35. Kết quả các testcase sau khi nạp ảnh gốc	47
Hình 36. Kết quả ảnh sau khi mã hóa AES-128	48
Hình 37. Kết quả test0 sau khi giải mã AES-128	49

DANH MỤC BẢNG BIỂU

Bảng 1. Danh mục thuật ngữ viết tắt sử dụng trong báo cáo.....	iv
Bảng 2. Mục tiêu chính của đề tài	2
Bảng 3. Phạm vi thực hiện và giới hạn của đề tài	3
Bảng 4. Thông số cơ bản của AES-128.....	5
Bảng 5. So sánh các phép biến đổi trong mã hóa và giải mã AES-128	13
Bảng 6. Cấu trúc bit của một pixel RGB565	14
Bảng 7. Quan hệ giữa kích thước ảnh, pixel, packet và block AES.....	14
Bảng 8. Thông số UART sử dụng trong hệ thống.....	15
Bảng 9. Cấu trúc packet UART trong hệ thống đề xuất.....	16
Bảng 10. Phân vùng SRAM trong hệ thống đề xuất	18
Bảng 11. Bố cục hiển thị VGA 640x480 trong hệ thống đề xuất.....	20
Bảng 12. Bảng phân tích yêu cầu hệ thống	23
Bảng 13. Các khối chính trong kiến trúc hệ thống đề xuất	24
Bảng 14. Bảng mô tả các thư mục chính trong thiết kế.....	27
Bảng 15. Các module trong khối UART image loader	28
Bảng 16. Tính toán dung lượng SRAM cho ảnh 320x240 RGB565	29
Bảng 17. Chính sách ưu tiên truy cập SRAM	30
Bảng 18. Nguồn và đích dữ liệu theo chế độ DMA	31
Bảng 19. Tín hiệu chính của AES wrapper	32
Bảng 20. Bảng trạng thái controller hệ thống.....	34
Bảng 21. Các thông tin debug nên hiển thị trên dashboard.....	35
Bảng 22. Môi trường thực hiện và kiểm thử	37
Bảng 23. Các nhóm mã nguồn RTL chính	38
Bảng 24. Testcase kiểm thử AES wrapper	39
Bảng 25. Testcase kiểm thử UART packet	40
Bảng 26. Testcase kiểm thử SRAM arbiter	41
Bảng 27. Testcase kiểm thử AES SRAM DMA.....	42
Bảng 28. Kịch bản kiểm thử tích hợp hệ thống.....	44

Bảng 29. Kết quả tài nguyên tổng hợp Quartus cho thiết kế chính	46
Bảng 30. Kết quả timing từ TimeQuest.....	46
Bảng 31. Bảng tổng hợp đánh giá kết quả hệ thống.....	49
Bảng 32. Hạn chế và hướng xử lý đề xuất.....	50
Bảng 33. Đối chiếu mục tiêu và mức độ hoàn thành.....	52
Bảng 34. Các hướng phát triển đề xuất	54

LỜI MỞ ĐẦU

Trong bối cảnh các hệ thống số ngày càng được ứng dụng rộng rãi trong truyền thông, xử lý ảnh và bảo vệ dữ liệu, việc nghiên cứu triển khai các thuật toán mã hóa trên phần cứng có ý nghĩa quan trọng đối với sinh viên ngành kỹ thuật máy tính. Khác với việc hiện thực thuật toán bằng phần mềm, triển khai trên FPGA yêu cầu người thiết kế phải quan tâm đồng thời đến thuật toán, luồng dữ liệu, tài nguyên phần cứng, tín hiệu điều khiển, bộ nhớ ngoài, giao tiếp ngoại vi và khả năng kiểm thử.

Đề tài này tập trung xây dựng hệ thống đề xuất, một hệ thống mã hóa và giải mã ảnh RGB565 bằng thuật toán AES-128 trên FPGA. Hệ thống nhận ảnh từ máy tính thông qua UART, ghi dữ liệu vào SRAM ngoài, xử lý ảnh theo từng block 128 bit bằng khối AES và hiển thị trực quan kết quả lên màn hình VGA. Cách tiếp cận này giúp quá trình kiểm chứng trở nên rõ ràng vì người dùng có thể quan sát ảnh gốc, ảnh sau mã hóa và ảnh sau giải mã trên cùng một màn hình.

Nhóm sinh viên xin gửi lời cảm ơn đến giảng viên hướng dẫn đã định hướng, góp ý và hỗ trợ trong quá trình thực hiện đề tài. Những kiến thức thu được từ quá trình triển khai hệ thống là cơ sở quan trọng để nhóm tiếp tục nghiên cứu các hệ thống số phức tạp hơn trong tương lai.

TP. Hồ Chí Minh, ngày 25 tháng 05 năm 2026

Nhóm sinh viên thực hiện

Bảng Phân Công

Sinh viên	Phân công việc	Phần trăm
Trần Duy Nhân	Lập trình, tham khảo và hoàn thiện thiết kế 2 bộ AES-128 Core gồm bộ mã hóa và bộ giải mã; xây dựng các module thành phần của AES như SubBytes/InvSubBytes, ShiftRows/InvShiftRows, MixColumns/InvMixColumns, AddRoundKey, KeyExpansion, Rcon, S-Box và Inv S-Box ; chuẩn hóa luồng xử lý block 128 bit; kiểm tra hoạt động mã hóa và giải mã bằng các vector kiểm thử; viết testbench cho từng khối AES và testbench tổng hợp cho AES encryption/decryption; phối hợp tích hợp AES core với khối wrapper để hệ thống có thể điều khiển bằng các tín hiệu start, busy, done, data input và data output.	100%
Nguyễn Hoàng Đăng Khoa	Lập trình, tham khảo và hoàn thiện các khối hệ thống xung quanh AES gồm giao tiếp SRAM, SRAM PHY, SRAM Arbiter, phân vùng bộ nhớ ảnh gốc/ảnh mã hóa/ảnh giải mã, VGA timing, VGA reader, dashboard/debug hiển thị, AES SRAM DMA, System Controllor/FSM tổng thể, UART image loader, packet receiver, CRC8, ACK/NACK , và phần mềm gửi ảnh từ máy tính xuống FPGA; thiết kế cơ chế đọc 8 pixel RGB565 để ghép thành block AES 128 bit, ghi kết quả trở lại SRAM; viết testbench cho	100%

	SRAM, Arbiter, DMA, UART packet, VGA timing và kiểm thử tích hợp toàn hệ thống.	
--	---	--

CHƯƠNG 1. MỞ ĐẦU

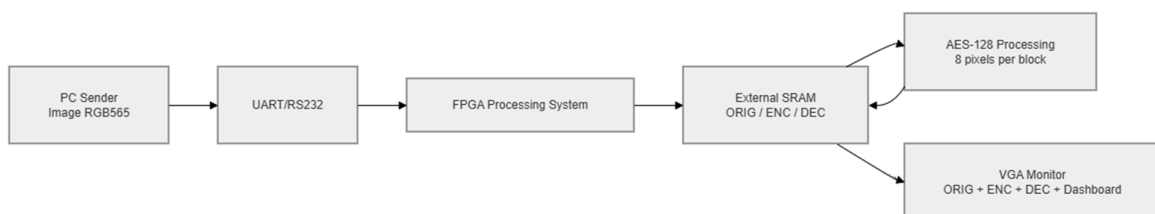
1.1 Lý do chọn đề tài

Ảnh số là một dạng dữ liệu trực quan, dễ quan sát và thường xuất hiện trong nhiều hệ thống xử lý thông tin. Khi ảnh được truyền qua kênh giao tiếp hoặc lưu trong bộ nhớ, nội dung ảnh có thể bị khai thác nếu không có cơ chế bảo vệ phù hợp. Vì vậy, việc áp dụng thuật toán mã hóa cho dữ liệu ảnh là một hướng nghiên cứu có tính thực tiễn, đồng thời phù hợp với mục tiêu học tập về hệ thống số và bảo mật dữ liệu.

Advanced Encryption Standard, gọi tắt là AES, là thuật toán mã hóa khối đối xứng được sử dụng phổ biến trong nhiều ứng dụng bảo mật. Với phiên bản AES-128, dữ liệu được xử lý theo từng block 128 bit và sử dụng khóa có kích thước 128 bit. Đặc điểm này phù hợp với thiết kế phần cứng vì các phép biến đổi trong AES như SubBytes, ShiftRows, MixColumns và AddRoundKey có thể được mô tả rõ ràng bằng Verilog HDL và kiểm thử bằng các vector chuẩn.

FPGA là nền tảng thích hợp để hiện thực các hệ thống xử lý dữ liệu theo luồng vì cho phép thiết kế phần cứng chuyên dụng, điều khiển song song và giao tiếp trực tiếp với ngoại vi như UART, SRAM và VGA. Trong đề tài này, FPGA không chỉ thực hiện thuật toán AES mà còn đảm nhiệm vai trò của một hệ thống xử lý ảnh hoàn chỉnh. Hệ thống cần nhận ảnh từ máy tính, lưu ảnh vào SRAM, đọc ảnh theo block, xử lý AES và hiển thị kết quả ra màn hình.

Xuất phát từ các yêu cầu trên, đề tài mã hóa và giải mã ảnh RGB565 bằng AES-128 trên FPGA được lựa chọn nhằm kết hợp nhiều nội dung cốt lõi của thiết kế số. Các nội dung này bao gồm mô tả phần cứng bằng Verilog HDL, thiết kế FSM, giao tiếp UART, điều khiển SRAM ngoài, xử lý dữ liệu bằng DMA, kiểm thử testbench và hiển thị VGA. Đề tài cũng giúp làm rõ sự khác biệt giữa việc viết thuật toán ở mức phần mềm và việc xây dựng một kiến trúc phần cứng có thể tổng hợp được.



Hình 1. Tổng quan bài toán mã hóa và giải mã ảnh trên FPGA

1.2 Mục tiêu đề tài

Mục tiêu tổng quát của đề tài là xây dựng một hệ thống phần cứng có khả năng mã hóa và giải mã ảnh RGB565 bằng AES-128 trên FPGA. Hệ thống cần có luồng dữ liệu hoàn chỉnh từ máy tính đến FPGA, từ FPGA đến SRAM, từ SRAM qua AES và từ SRAM đến VGA. Kết quả cuối cùng cần thể hiện được ảnh gốc, ảnh mã hóa và ảnh giải mã để người dùng có thể đánh giá trực quan tính đúng của quá trình xử lý.

Về mặt thiết kế phần cứng, đề tài cần tổ chức các module RTL theo kiến trúc phân cấp rõ ràng. Mỗi module đảm nhiệm một chức năng riêng như nhận UART, kiểm tra packet, ghi SRAM, phân xử truy cập SRAM, điều khiển DMA, bọc lõi AES, tạo timing VGA và hiển thị dashboard trạng thái. Cách tổ chức này giúp hệ thống dễ mô phỏng, dễ tích hợp và dễ mở rộng trong các phiên bản sau.

Về mặt kiểm thử, đề tài cần xây dựng các testbench độc lập cho những module quan trọng. Các testbench phải kiểm tra được chức năng AES với known-answer vector, kiểm tra packet UART và CRC8, kiểm tra ghi SRAM, kiểm tra DMA pack và unpack dữ liệu ảnh, kiểm tra phân xử SRAM và kiểm tra timing VGA. Ngoài ra, hệ thống cần có một smoke test ở mức tích hợp để xác nhận các khối có thể phối hợp với nhau theo đúng luồng thiết kế.

Bảng 2. Mục tiêu chính của đề tài

Nội dung mục tiêu	Tiêu chí đánh giá
Xây dựng hệ thống nạp ảnh 320x240 RGB565 từ máy tính qua UART.	FPGA nhận đủ packet, kiểm tra CRC8 và ghi đúng dữ liệu ảnh vào SRAM.
Tổ chức bộ nhớ SRAM thành các vùng ảnh gốc, ảnh mã hóa và ảnh giải mã.	Địa chỉ vùng nhớ rõ ràng, không chồng lấn và phù hợp dung lượng ảnh.
Tích hợp lõi AES-128 với giao diện start, busy và done.	AES mã hóa và giải mã đúng known-answer vector.
Thiết kế DMA đọc ghi 8 pixel RGB565 cho mỗi block AES 128 bit.	DMA đọc đúng thứ tự pixel, pack block, nhận kết quả AES và ghi lại SRAM.
Hiển thị ảnh và dashboard trạng thái qua VGA 640x480.	Màn hình thể hiện được vùng ORIG, ENC, DEC và trạng thái debug.
Xây dựng testbench và tổng hợp thiết kế trên Quartus.	Các self-test đạt và project tổng hợp thành công trên Cyclone II.

1.3 Đối tượng và phạm vi thực hiện

Đối tượng nghiên cứu của đề tài là một hệ thống xử lý ảnh được mô tả bằng Verilog HDL. Hệ thống hướng đến việc xử lý ảnh tĩnh kích thước 320x240, định dạng RGB565, trong đó mỗi pixel được biểu diễn bằng một word 16 bit. Ảnh được truyền từ máy tính xuống FPGA theo từng packet UART và được lưu trong SRAM ngoài trước khi thực hiện mã hóa hoặc giải mã.

Phạm vi thuật toán tập trung vào AES-128 với block dữ liệu 128 bit và khóa 128 bit. Báo cáo trình bày các phép biến đổi cơ bản trong mã hóa và giải mã AES-128, nhưng không mở rộng sang các chế độ mã hóa phức tạp như CBC, CTR hoặc GCM. Trong phiên bản hiện tại, hệ thống xử lý từng block độc lập theo hướng phù hợp với việc minh họa phần cứng và hiển thị trực quan trên VGA.

Phạm vi phần cứng tập trung vào các khối chức năng chính của hệ thống, bao gồm khối kết nối phần cứng mức cao, khối điều khiển trung tâm, khối nạp ảnh qua UART, khối phân xử SRAM, khối DMA xử lý ảnh qua AES, khối chuẩn hóa giao diện lỗi AES và khối hiển thị VGA. Báo cáo chỉ trình bày các thành phần phục vụ trực tiếp cho kiến trúc đề xuất, không đi sâu vào các phiên bản thử nghiệm hoặc các nhánh cài đặt không còn thuộc luồng thiết kế chính.

Phạm vi kiểm thử bao gồm mô phỏng chức năng bằng ModelSim-Altera, self-checking testbench, waveform kiểm tra và kết quả tổng hợp bằng Quartus II. Việc đánh giá bảo mật ở mức mật mã học chuyên sâu không nằm trong phạm vi của đề tài. Đề tài cũng chưa triển khai cơ chế xác thực ảnh, chưa hỗ trợ truyền ảnh có kích thước động và chưa có DMA compare pass đầy đủ để so sánh toàn bộ ảnh giải mã với ảnh gốc trên phần cứng.

Bảng 3. Phạm vi thực hiện và giới hạn của đề tài

Nhóm nội dung	Trong phạm vi	Ngoài phạm vi hoặc hướng mở rộng
Dữ liệu ảnh	Ảnh RGB565 kích thước 320x240, truyền dưới dạng byte qua UART.	Ảnh kích thước động, ảnh nén hoặc ảnh nhiều định dạng màu.
Thuật toán	AES-128 theo block 128 bit, có mã hóa và giải mã.	Các chế độ CBC, CTR, GCM hoặc xác thực dữ liệu.
Phần cứng	UART, SRAM, AES DMA, VGA và controller trên FPGA.	CPU mềm Nios II, Ethernet, USB hoặc lưu trữ SD card.
Kiểm thử	Self-test module, smoke test, waveform và tổng hợp Quartus.	Đánh giá mật mã học nâng cao hoặc đo công suất tiêu thụ chi tiết.

1.4 Phương pháp thực hiện

Quá trình thực hiện đề tài được triển khai theo hướng phân tích từ yêu cầu hệ thống đến kiến trúc module. Trước tiên, nhóm khảo sát thuật toán AES-128, định dạng ảnh RGB565 và các ngoại vi cần sử dụng trên FPGA. Sau đó, hệ thống được chia thành các khối chức năng độc lập để thuận tiện cho việc mô tả RTL, mô phỏng và kiểm thử.

Đối với phần truyền ảnh, nhóm xây dựng cơ chế packet UART gồm header, payload, CRC8 và phản hồi ACK hoặc NACK. Phía máy tính chịu trách nhiệm chuyển đổi ảnh sang RGB565, chia ảnh thành các packet 256 byte và gửi tuần tự xuống FPGA. Phía FPGA nhận byte, kiểm tra packet, gom hai byte thành một word RGB565 và ghi vào vùng ảnh gốc trong SRAM.

Đối với phần xử lý AES, hệ thống sử dụng lõi mã hóa và giải mã AES-128 như một khối xử lý trung tâm. Lõi này được bao bọc bởi một lớp giao diện điều khiển nhằm chuẩn hóa các tín hiệu start, busy và done, nhờ đó DMA có thể điều khiển theo dạng handshake. DMA đọc 8 pixel từ SRAM để tạo một block 128 bit, kích hoạt AES và ghi kết quả về vùng ảnh mã hóa hoặc giải mã.

Đối với phần đánh giá, nhóm sử dụng testbench để kiểm tra từng module và dùng Quartus để tổng hợp thiết kế. Kết quả tổng hợp được dùng để đánh giá mức sử dụng tài nguyên, timing slack và khả năng đáp ứng xung clock 50 MHz. Các placeholder hình trong báo cáo được bố trí để người thực hiện có thể thay bằng waveform, ảnh chụp VGA, sơ đồ kiến trúc và ảnh minh họa thực tế.

CHƯƠNG 2. CƠ SỞ LÝ THUYẾT VÀ CÔNG NGHỆ LIÊN QUAN

2.1 Tổng quan về thuật toán AES-128

Advanced Encryption Standard là chuẩn mã hóa khối đối xứng do NIST công bố trong FIPS 197. AES được thiết kế để mã hóa và giải mã dữ liệu bằng cùng một khóa bí mật. Phiên bản AES-128 sử dụng khóa 128 bit, block dữ liệu 128 bit và thực hiện 10 vòng biến đổi. Chuẩn FIPS 197 được cập nhật về mặt biên tập vào năm 2023 nhưng không thay đổi kỹ thuật của thuật toán so với bản ban đầu [1], [2].

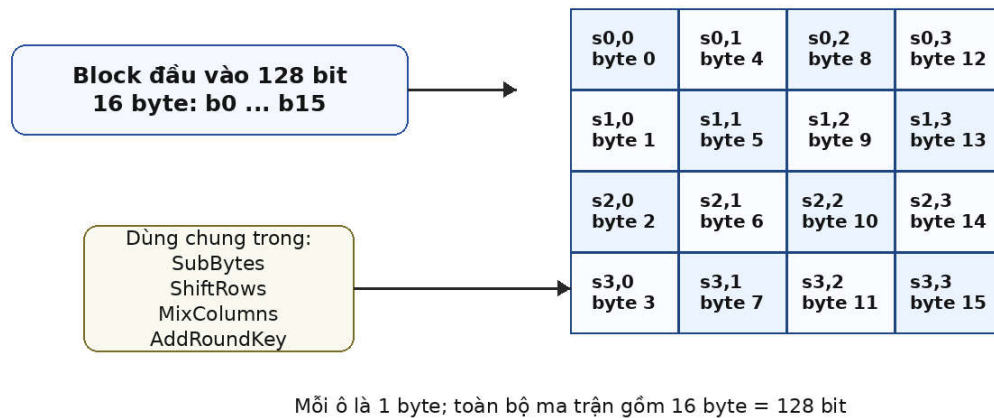
Trong AES, block dữ liệu 128 bit được tổ chức thành ma trận trạng thái gồm 4 hàng và 4 cột, mỗi phần tử là một byte. Cách biểu diễn này giúp mô tả rõ các phép biến đổi như thay thế byte, dịch hàng, trộn cột và cộng khóa vòng. Trong phần cứng, ma trận trạng thái có thể được biểu diễn bằng vector 128 bit, trong đó việc chọn byte được thực hiện bằng part-select hoặc ánh xạ cố định theo thứ tự byte.

AES-128 có ba nhóm xử lý chính trong quá trình mã hóa. Bước đầu tiên là AddRoundKey với khóa ban đầu. Chín vòng tiếp theo gồm SubBytes, ShiftRows, MixColumns và AddRoundKey. Vòng cuối cùng bỏ qua MixColumns và chỉ thực hiện SubBytes, ShiftRows và AddRoundKey. Cách tổ chức này được sử dụng trong thiết kế hệ thống đề xuất thông qua các module AES core gốc và wrapper điều khiển.

Đối với giải mã, quá trình xử lý sử dụng các phép biến đổi nghịch đảo gồm InvShiftRows, InvSubBytes và InvMixColumns, đồng thời sử dụng các khóa vòng theo thứ tự ngược lại. Bài viết tham khảo của Nguyễn Quân trình bày ngắn gọn quá trình mã hóa và giải mã AES-128 theo hướng dễ liên hệ với thiết kế RTL, trong đó nhấn mạnh các chức năng AddRoundKey, SubBytes, ShiftRows, MixColumns và KeyExpansion [3], [4].

Bảng 4. Thông số cơ bản của AES-128

Thông số	Giá trị	Ý nghĩa trong đề tài
Kích thước block	128 bit	Mỗi lần AES xử lý tương đương 8 pixel RGB565.
Kích thước khóa	128 bit	Thiết kế sử dụng khóa demo cố định 2b7e151628aed2a6abf7158809cf4f3c.
Số vòng mã hóa	10 vòng	Gồm 1 bước khởi tạo, 9 vòng đầy đủ và 1 vòng cuối.
Dạng dữ liệu	Ma trận 4x4 byte	Giúp mô tả SubBytes, ShiftRows và MixColumns.
Kiểu thuật toán	Mã hóa khối đối xứng	Cùng một khóa được dùng để phục vụ mã hóa và giải mã.



Hình 2. Biểu diễn block 128 bit thành ma trận trạng thái AES

2.1.1 Khái niệm mã hóa khối và AES-128

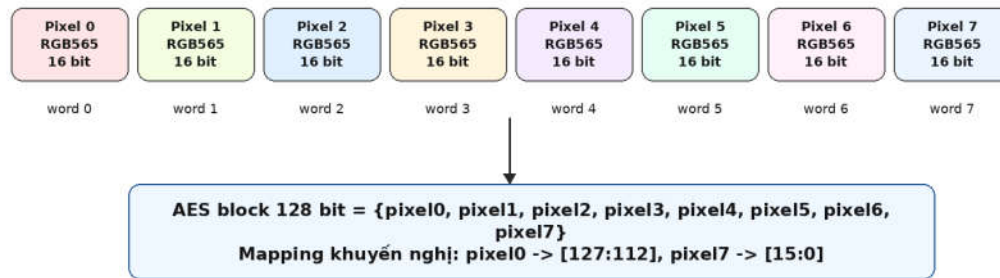
Mã hóa khối là phương pháp mã hóa dữ liệu theo từng khối có kích thước cố định. Với AES, kích thước khối luôn là 128 bit. Khi dữ liệu đầu vào lớn hơn một block, dữ liệu sẽ được chia thành nhiều block và xử lý tuần tự hoặc song song tùy kiến trúc. Trong đề tài này, ảnh RGB565 được xem như một chuỗi dữ liệu nhị phân, sau đó được chia thành các block 128 bit để đưa vào lõi AES.

Việc sử dụng mã hóa khối trong xử lý ảnh có một ưu điểm quan trọng là thuận tiện cho phần cứng. Mỗi block có kích thước cố định nên DMA có thể thiết kế theo chu kỳ đọc cố định. Với RGB565, mỗi pixel có 16 bit, do đó một block AES 128 bit chứa đúng 8 pixel. Quan hệ này giúp đơn giản hóa bộ gom dữ liệu và bộ tách dữ liệu trong DMA.

2.1.2 Cấu trúc dữ liệu 128 bit trong AES

Trong thiết kế hệ thống đề xuất, dữ liệu vào của AES wrapper là block dữ liệu vào[127:0] và dữ liệu ra là block dữ liệu ra[127:0]. Cách mô tả này giúp wrapper giữ giao diện đơn giản, trong khi các module con của AES thực hiện ánh xạ byte nội bộ. Khi DMA đọc SRAM, 8 word 16 bit được ghép lại thành một vector 128 bit. Khi AES xử lý xong, vector 128 bit được tách lại thành 8 word để ghi về SRAM.

Điểm cần chú ý khi thiết kế phần cứng là thứ tự byte và thứ tự word phải thống nhất từ công cụ gửi ảnh, UART packet writer, SRAM và DMA. Nếu thứ tự byte bị đảo, ảnh hiển thị có thể sai màu hoặc kết quả AES không thể giải mã chính xác. Vì vậy, báo cáo cần trình bày rõ quan hệ giữa byte truyền qua UART, word RGB565 trong SRAM và block 128 bit trong AES.

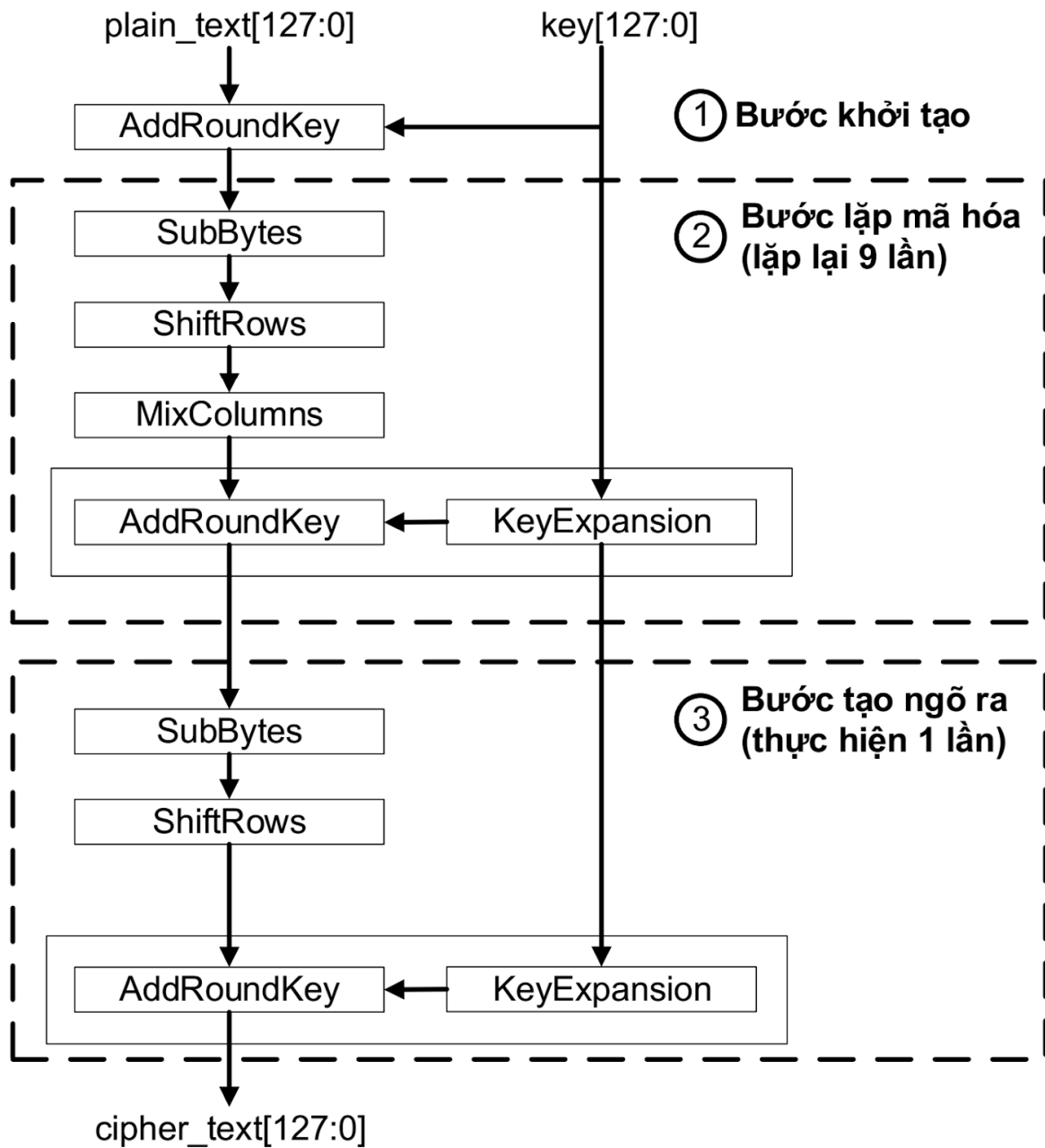


Hình 3. Cách ghép 8 pixel RGB565 thành một block AES 128 bit

2.1.3 Tổng quan quá trình mã hóa AES-128

Quá trình mã hóa AES-128 bắt đầu bằng phép AddRoundKey giữa plaintext và khóa ban đầu. Sau đó, dữ liệu đi qua 9 vòng lặp gồm SubBytes, ShiftRows, MixColumns và AddRoundKey. Cuối cùng, dữ liệu đi qua vòng cuối không có MixColumns để tạo ciphertext. Trong phần cứng, các phép biến đổi này có thể được mô tả thành các module tổ hợp hoặc tuần tự tùy cách tối ưu tài nguyên và độ trễ.

Trong thiết kế hệ thống đề xuất, lõi AES encryption được giữ nguyên từ core gốc. Việc giữ nguyên core giúp hạn chế thay đổi thuật toán và giảm rủi ro sai khác so với thiết kế ban đầu. Phần được cải tiến nằm ở lớp wrapper và lớp hệ thống, nơi lõi AES được tích hợp vào DMA, SRAM và controller để có thể xử lý ảnh theo block.

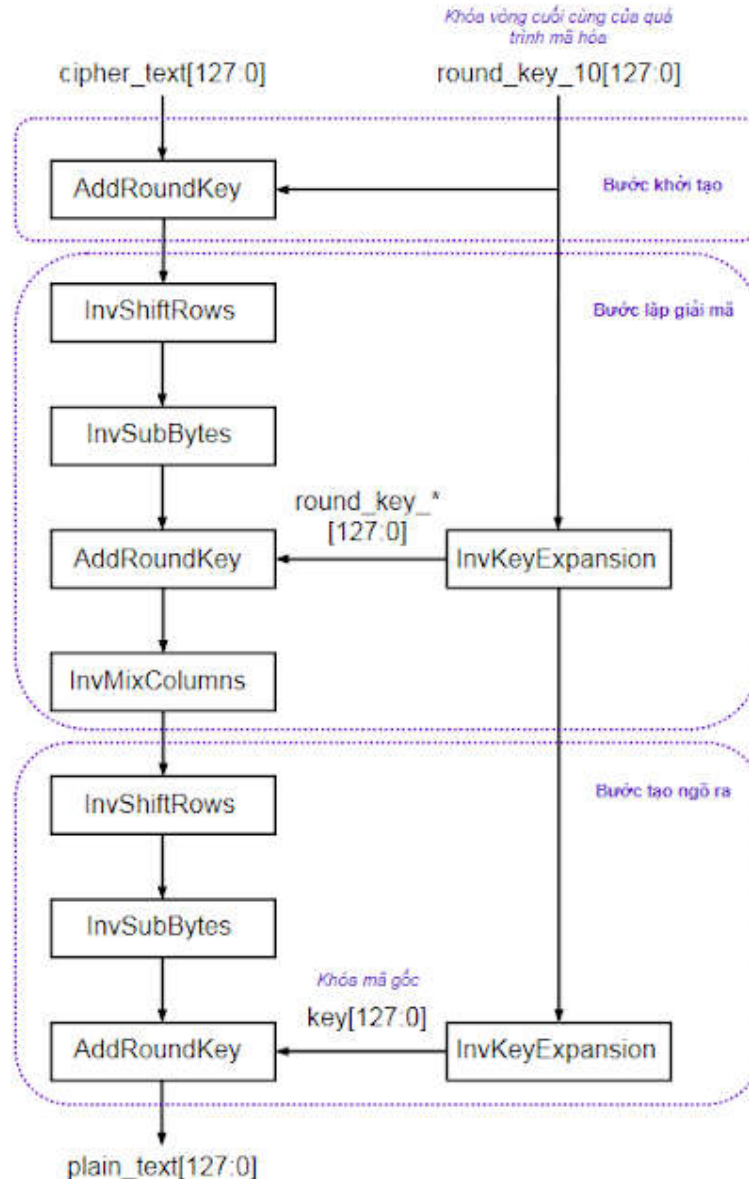


Hình 4. Quá trình mã hóa AES-128

2.1.4 Tổng quan quá trình giải mã AES-128

Giải mã AES-128 là quá trình khôi phục plaintext từ ciphertext bằng khóa tương ứng. Các phép biến đổi được sử dụng trong giải mã gồm AddRoundKey, InvShiftRows, InvSubBytes và InvMixColumns. Khác với mã hóa, các khóa vòng được sử dụng theo thứ tự ngược, bắt đầu từ khóa vòng cuối và kết thúc bằng khóa ban đầu.

Trong hệ thống đề xuất, chế độ giải mã được điều khiển bằng tín hiệu decrypt. Khi tín hiệu này bằng 1, wrapper kích hoạt lõi AES decryption thay vì lõi encryption. DMA không cần biết chi tiết bên trong AES mà chỉ cần đưa block vào, chờ tín hiệu done và nhận block kết quả. Cách tách lớp này giúp phần DMA có thể dùng chung cho cả mã hóa và giải mã.



Hình 5. Quá trình giải mã AES-128

2.2 Các phép biến đổi trong mã hóa AES-128

2.2.1 AddRoundKey

AddRoundKey là phép XOR giữa ma trận trạng thái và khóa vòng. Đây là phép biến đổi duy nhất trong AES trực tiếp sử dụng khóa tại từng vòng. Vì XOR là phép toán đơn

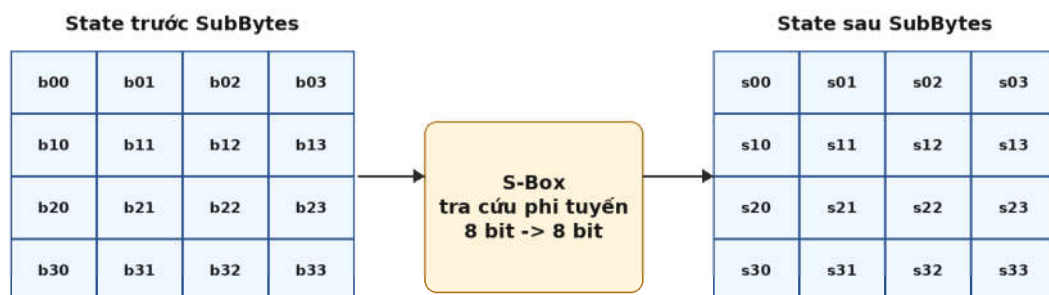
giản ở mức phần cứng, AddRoundKey có thể được hiện thực bằng các cổng XOR song song trên 128 bit.

Trong thiết kế RTL, AddRoundKey thường là module tổ hợp nhận dữ liệu trạng thái và round key, sau đó xuất kết quả XOR. Module khối AddRoundKey trong thiết kế hệ thống đề xuất thực hiện chức năng này và được sử dụng bởi cả hướng mã hóa và giải mã. Vai trò của phép này là đưa thông tin khóa vào dữ liệu ở mỗi vòng xử lý.

2.2.2 SubBytes và S-Box

SubBytes là phép thay thế phi tuyến từng byte của ma trận trạng thái bằng bảng S-Box. Mục tiêu của phép biến đổi này là tạo tính phi tuyến cho thuật toán, làm cho quan hệ giữa plaintext, key và ciphertext khó dự đoán hơn. Trong phần cứng, S-Box có thể được hiện thực bằng bảng tra cứu hoặc logic tổ hợp tối ưu.

Thiết kế hệ thống đề xuất sử dụng module khối S-Box từng byte cho từng byte và module khối SubBytes để áp dụng S-Box cho toàn bộ trạng thái 128 bit. Khi viết báo cáo, nên chèn hình bảng S-Box hoặc sơ đồ 16 byte đi qua 16 S-Box song song để minh họa rõ tính chất xử lý từng byte.



Trong RTL có thể triển khai bằng ROM/lookup table hoặc logic tổ hợp. 16 byte có thể đi qua 16 S-Box song song.

Hình 6. Minh họa phép SubBytes và bảng S-Box

2.2.3 ShiftRows

ShiftRows là phép dịch vòng các hàng trong ma trận trạng thái. Hàng đầu tiên giữ nguyên, hàng thứ hai dịch trái một byte, hàng thứ ba dịch trái hai byte và hàng thứ tư dịch

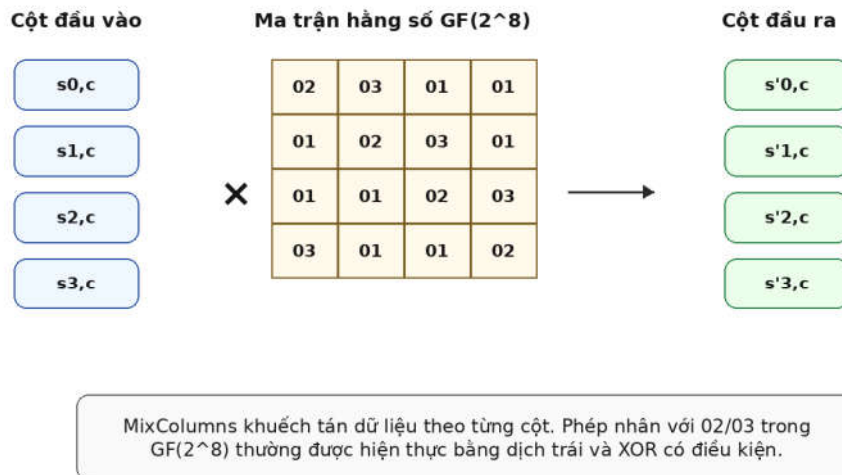
trái ba byte. Phép biến đổi này không làm thay đổi giá trị byte mà chỉ thay đổi vị trí, từ đó tạo sự khuếch tán giữa các cột trước khi đi vào MixColumns.

Ở mức RTL, ShiftRows chủ yếu là ánh xạ lại vị trí các byte trong ma trận trạng thái. Do đó, phép biến đổi này có thể được mô tả bằng các kết nối tổ hợp giữa đầu vào và đầu ra. Đây là phép biến đổi có chi phí tài nguyên thấp nhưng giữ vai trò quan trọng trong cấu trúc vòng AES.

2.2.4 MixColumns

MixColumns thực hiện biến đổi tuyến tính trên từng cột của ma trận trạng thái trong trường hữu hạn $GF(2^8)$. Mỗi cột gồm 4 byte được nhân với một ma trận hằng số theo quy định của AES. Phép nhân trong $GF(2^8)$ có thể quy đổi về dịch trái và XOR có điều kiện, vì vậy phù hợp để mô tả bằng logic tổ hợp trong Verilog.

Trong thiết kế phần cứng, MixColumns có thể được chia theo từng cột của ma trận trạng thái. Cách tách theo cột giúp quá trình mô tả, kiểm thử và đối chiếu với lý thuyết trở nên rõ ràng hơn. Đối với báo cáo, cần giải thích rằng vòng cuối của mã hóa không thực hiện MixColumns, đây là điểm khác biệt quan trọng so với các vòng chính.



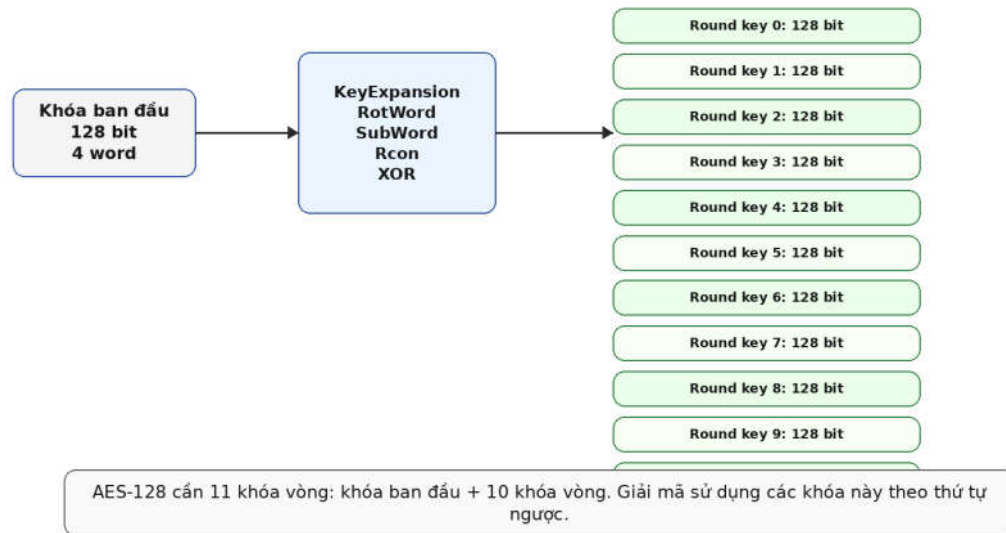
Hình 7. Minh họa phép MixColumns trên một cột trạng thái

2.2.5 KeyExpansion, RotWord, SubWord và Rcon

KeyExpansion tạo ra các khóa vòng từ khóa ban đầu. Với AES-128, khóa ban đầu gồm 4 word 32 bit và toàn bộ quá trình mở rộng tạo ra 44 word, tương ứng với khóa ban đầu và

10 khóa vòng. Các thao tác quan trọng trong KeyExpansion gồm RotWord, SubWord và XOR với Rcon.

Trong thiết kế phần cứng, KeyExpansion có thể được thực hiện trước rồi lưu khóa vòng, hoặc thực hiện theo từng vòng tùy kiến trúc. Thiết kế hệ thống đề xuất giữ cách tổ chức trong lõi AES gốc. Báo cáo cần trình bày KeyExpansion ở mức nguyên lý vừa đủ để người đọc hiểu tại sao AES cần khóa vòng và vì sao giải mã phải dùng khóa theo thứ tự ngược.



Hình 8. Quá trình KeyExpansion trong AES-128

2.3 Các phép biến đổi trong giải mã AES-128

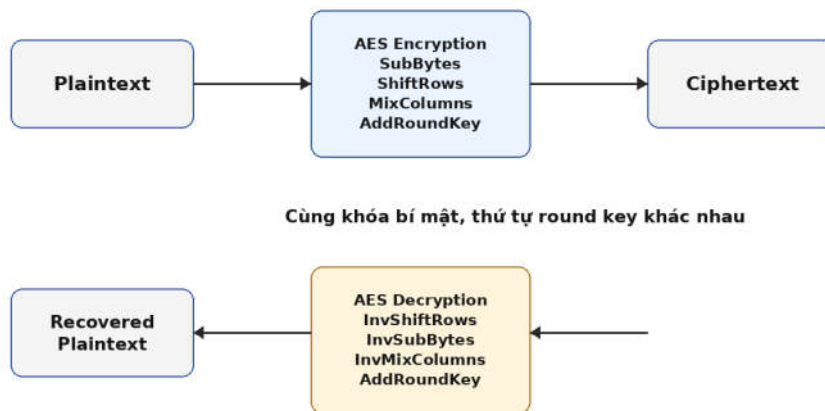
Các phép biến đổi trong giải mã có quan hệ nghịch đảo với các phép biến đổi trong mã hóa. InvShiftRows đảo lại vị trí các hàng, InvSubBytes sử dụng bảng S-Box đảo, InvMixColumns dùng ma trận nghịch đảo và AddRoundKey vẫn là phép XOR với khóa vòng. Vì XOR tự nghịch đảo, AddRoundKey có thể dùng chung cho cả mã hóa và giải mã.

Khi viết báo cáo, phần giải mã không nên tách rời khỏi thiết kế hệ thống. Nội dung cần nhấn mạnh rằng cùng một DMA có thể phục vụ cả hai hướng xử lý, còn lựa chọn encryption hay decryption được quyết định bởi tín hiệu điều khiển decrypt. Điều này làm giảm độ phức tạp tích hợp vì các khối SRAM và VGA không cần biết dữ liệu đang ở chế độ mã hóa hay giải mã.

Bảng 5. So sánh các phép biến đổi trong mã hóa và giải mã AES-128

Bảng 5. So sánh các phép biến đổi trong mã hóa và giải mã AES-128

Mã hóa	Giải mã tương ứng	Vai trò
SubBytes	InvSubBytes	Thay thế byte bằng S-Box hoặc S-Box đảo.
ShiftRows	InvShiftRows	Thay đổi vị trí byte theo hàng của ma trận trạng thái.
MixColumns	InvMixColumns	Khuếch tán dữ liệu theo từng cột trong GF(2 ⁸).
AddRoundKey	AddRoundKey	XOR trạng thái với khóa vòng, dùng chung cho hai hướng.
KeyExpansion	Thứ tự khóa vòng ngược	Tạo và sử dụng khóa vòng đúng thứ tự cho từng hướng.

**Hình 9. Mối quan hệ giữa AES encryption và AES decryption**

2.4 Biểu diễn ảnh RGB565

Ảnh số được biểu diễn bằng tập hợp các pixel, trong đó mỗi pixel chứa thông tin màu. Với không gian màu RGB, mỗi pixel gồm ba thành phần đỏ, xanh lá và xanh dương. Định dạng RGB565 sử dụng 16 bit cho mỗi pixel, gồm 5 bit cho màu đỏ, 6 bit cho màu xanh lá và 5 bit cho màu xanh dương. Sở dĩ màu xanh lá có 6 bit là vì mắt người nhạy hơn với thành phần này so với đỏ và xanh dương.

Trong đề tài, RGB565 là lựa chọn phù hợp vì mỗi pixel vừa đúng một word 16 bit của SRAM. Cách tổ chức này giúp giảm độ phức tạp khi đọc ghi bộ nhớ vì mỗi địa chỉ SRAM tương ứng với một pixel. Ngoài ra, độ sâu màu 16 bit vẫn đủ để hiển thị ảnh demo trực quan trên VGA, trong khi dung lượng bộ nhớ thấp hơn so với RGB888.

Bảng 6. Cấu trúc bit của một pixel RGB565

Thành phần	Số bit	Vị trí bit trong word 16 bit	Ý nghĩa
Red	5	[15:11]	Cường độ màu đỏ.
Green	6	[10:5]	Cường độ màu xanh lá.
Blue	5	[4:0]	Cường độ màu xanh dương.



1 pixel = 16 bit = 1 word SRAM

Hình 10. Cấu trúc định dạng RGB565

Ảnh đầu vào trong thiết kế có kích thước 320x240, tương ứng 76.800 pixel. Vì mỗi pixel có 16 bit, tổng dung lượng ảnh là 153.600 byte. Với cách packet hóa của hệ thống, mỗi packet UART chứa 256 byte dữ liệu, tương ứng 128 pixel RGB565. Do đó, toàn bộ ảnh cần đúng 600 packet.

Khi đưa ảnh vào AES, mỗi block 128 bit tương ứng 8 pixel. Một dòng ảnh có 320 pixel nên gồm 40 block AES. Toàn bộ ảnh có 240 dòng nên cần 9.600 block AES. Đây là quan hệ quan trọng cần trình bày vì nó giải thích vì sao DMA trong thiết kế có bộ đếm block, bộ đếm dòng và bộ đếm nhóm cột.

Bảng 7. Quan hệ giữa kích thước ảnh, pixel, packet và block AES

Đại lượng	Giá trị	Diễn giải
Kích thước ảnh	320 x 240	Tổng số pixel là 76.800.
Dung lượng mỗi pixel	16 bit	Một pixel RGB565 bằng một word SRAM.
Dung lượng ảnh	153.600 byte	Bằng 76.800 word 16 bit.
Dung lượng mỗi packet	256 byte	Mỗi packet chứa 128 pixel.
Số packet toàn ảnh	600	153.600 chia cho 256.
Số pixel mỗi block AES	8	128 bit chia cho 16 bit.
Số block AES toàn ảnh	9.600	40 block mỗi dòng nhân 240 dòng.

2.5 Giao tiếp UART trên kit FPGA

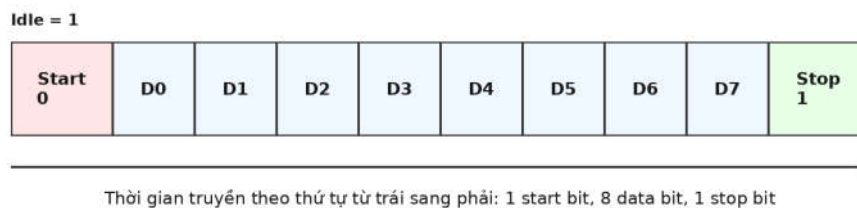
UART là chuẩn giao tiếp nối tiếp bất đồng bộ, thường được sử dụng để truyền dữ liệu giữa máy tính và thiết bị nhúng. Một khung UART cơ bản gồm start bit, các bit dữ liệu và stop bit. Trong hệ thống đề xuất, UART được dùng để truyền ảnh từ máy tính xuống FPGA vì dễ triển khai, dễ kiểm thử và phù hợp với mục tiêu demo phần cứng.

Thiết kế sử dụng tốc độ truyền 115200 baud theo cấu hình phổ biến 8N1, nghĩa là 8 bit dữ liệu, không parity và 1 stop bit. Với ảnh 153.600 byte, thời gian truyền lý thuyết ở 115200 baud khoảng hơn 13 giây nếu chỉ xét dữ liệu UART 10 bit cho mỗi byte. Khi tính thêm overhead ACK, phần mềm gửi và độ trễ hệ thống, thời gian thực tế có thể nằm trong khoảng 14 đến 25 giây theo ghi chú trong thiết kế.

RS232 là lớp vật lý thường dùng cho giao tiếp nối tiếp trên các kit FPGA đời cũ. Trong báo cáo, UART và RS232 nên được trình bày cùng nhau theo hướng UART là logic truyền nhận byte trong FPGA, còn RS232 là giao tiếp vật lý hoặc adapter kết nối với máy tính. Điều này giúp tránh nhầm lẫn giữa giao thức logic và chuẩn mức điện áp.

Bảng 8. Thông số UART sử dụng trong hệ thống

Thông số	Giá trị	Vai trò
Baudrate	115200	Tốc độ truyền byte từ PC xuống FPGA.
Data bit	8	Mỗi byte dữ liệu ảnh được truyền 8 bit.
Parity	None	Không dùng bit kiểm tra parity.
Stop bit	1	Kết thúc một khung UART.
Byte order	High byte trước, low byte sau	Dùng để ghép hai byte thành một pixel RGB565.



Hình 11. Cấu trúc khung truyền UART 8N1

2.6 Cơ chế packet, CRC8 và ACK/NACK

Truyền ảnh trực tiếp dưới dạng một chuỗi byte dài có thể gây khó khăn trong việc phát hiện lỗi hoặc xác định vị trí dữ liệu. Vì vậy, hệ thống đề xuất đóng gói dữ liệu ảnh thành

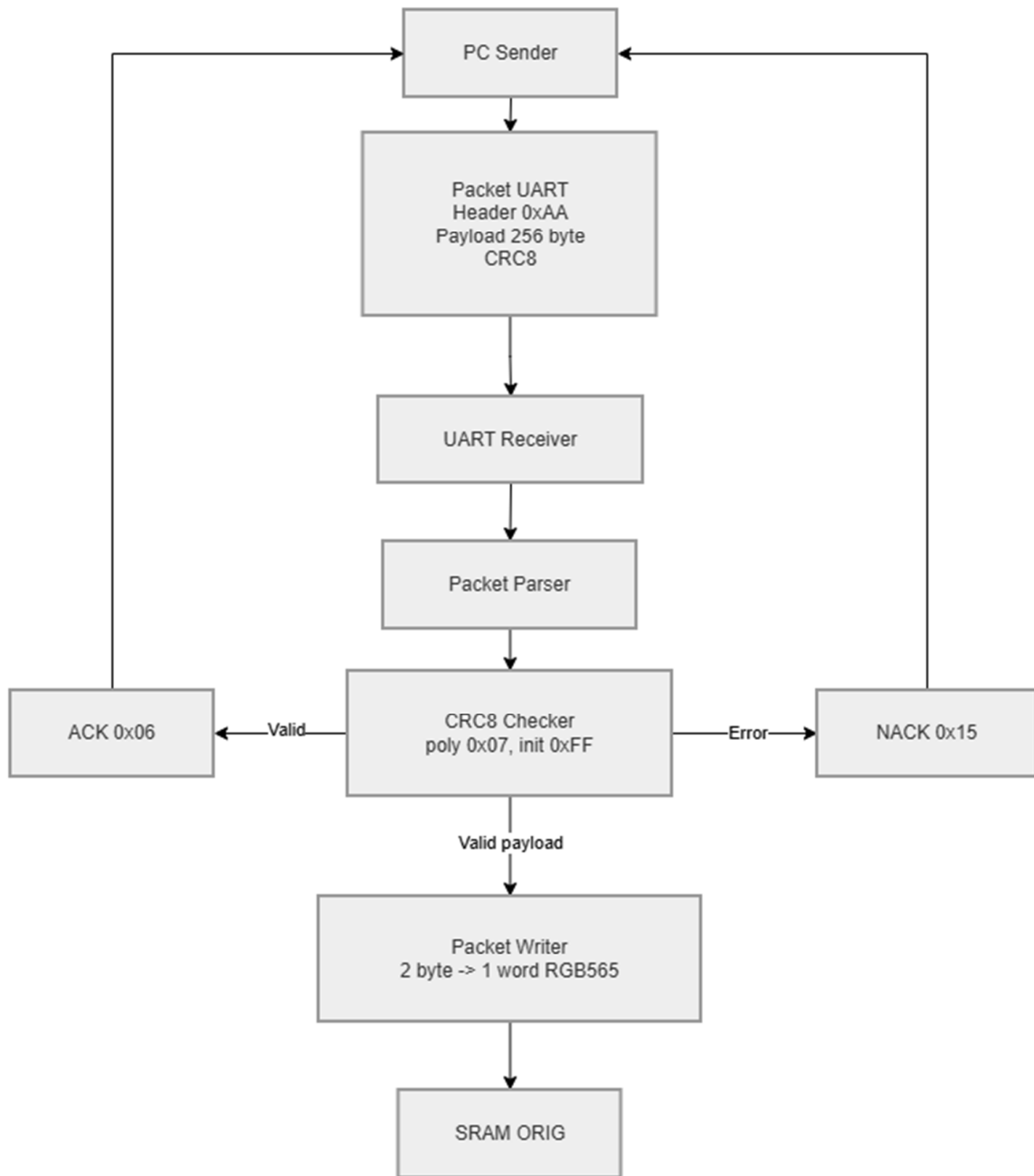
các packet có kích thước cố định. Mỗi packet gồm header, payload 256 byte và CRC8. Sau khi nhận và kiểm tra packet, FPGA phản hồi ACK nếu packet hợp lệ hoặc NACK nếu packet sai.

CRC8 được sử dụng để kiểm tra lỗi trên payload của packet. Trong thiết kế, CRC8 dùng đa thức $0x07$, giá trị khởi tạo $0xFF$ và chỉ tính trên vùng DATA. Cách kiểm tra này không thay thế cho cơ chế bảo mật nhưng đủ để phát hiện các lỗi truyền thường gặp trong quá trình nạp ảnh qua UART.

Cơ chế ACK/NACK giúp phần mềm gửi ảnh biết được packet đã được FPGA nhận đúng hay cần truyền lại. Đây là một điểm quan trọng đối với demo phần cứng vì nếu chỉ gửi liên tục mà không có phản hồi, một byte lỗi có thể làm sai toàn bộ ảnh trong SRAM. Với ACK/NACK, hệ thống có thể tăng độ tin cậy trong phạm vi truyền ảnh qua UART.

Bảng 9. Cấu trúc packet UART trong hệ thống đề xuất

Trường	Kích thước	Giá trị hoặc mô tả
HEADER	1 byte	$0xAA$, dùng để đánh dấu bắt đầu packet.
DATA	256 byte	Dữ liệu ảnh RGB565 theo byte, high byte trước.
CRC8	1 byte	CRC8 poly $0x07$, init $0xFF$, tính trên DATA.
ACK	1 byte	$0x06$, FPGA trả về khi packet hợp lệ.
NACK	1 byte	$0x15$, FPGA trả về khi packet lỗi.



Hình 12. Sơ đồ cấu trúc packet UART và cơ chế ACK/NACK

2.7 Bộ nhớ ngoài SRAM trên kit FPGA

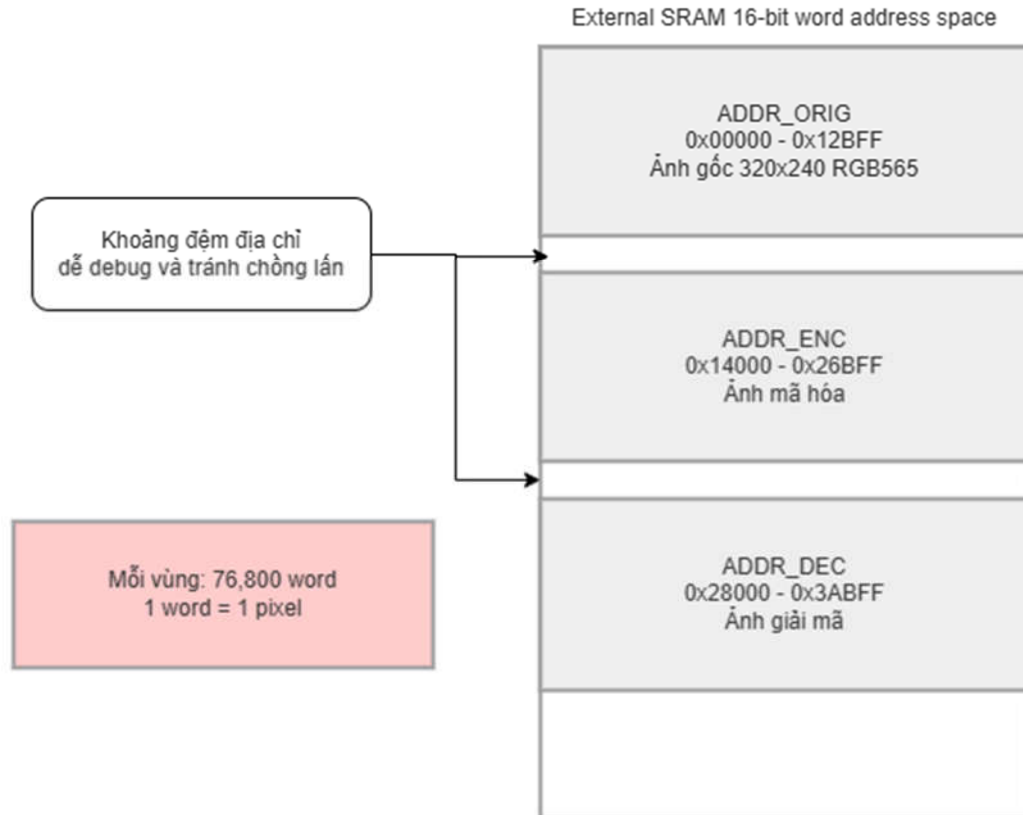
SRAM ngoài đóng vai trò là vùng lưu trữ chính của hệ thống. Ảnh 320x240 RGB565 cần 76.800 word 16 bit cho mỗi bản ảnh. Hệ thống lưu ba bản ảnh gồm ảnh gốc, ảnh mã hóa và ảnh giải mã, do đó cần 230.400 word 16 bit. Dung lượng này phù hợp với SRAM ngoài nhưng không phù hợp để lưu hoàn toàn bằng bộ nhớ trong nếu dùng FPGA Cyclone II có tài nguyên hạn chế.

SRAM có bus địa chỉ, bus dữ liệu hai chiều và các tín hiệu điều khiển như WE_N, OE_N, CE_N, UB_N và LB_N. Vì bus dữ liệu là inout, module khối giao tiếp vật lý SRAM 16 bit chịu trách nhiệm điều khiển hướng dữ liệu khi đọc và ghi. Các module khác không truy cập trực tiếp chân SRAM mà đi qua arbiter và PHY để giữ kiến trúc rõ ràng.

Trong hệ thống, SRAM bị chia sẻ bởi nhiều master. UART loader cần ghi ảnh gốc khi nạp ảnh. AES DMA cần đọc và ghi khi mã hóa hoặc giải mã. VGA cần đọc SRAM để hiển thị ảnh. Do đó, hệ thống cần arbiter để quyết định master nào được quyền truy cập bus tại từng thời điểm.

Bảng 10. Phân vùng SRAM trong hệ thống đề xuất

Vùng nhớ	Địa chỉ bắt đầu	Dung lượng	Chức năng
ORIG	18'h00000	76.800 word	Lưu ảnh gốc nhận từ PC hoặc ROM debug.
ENC	18'h14000	76.800 word	Lưu ảnh sau khi mã hóa AES-128.
DEC	18'h28000	76.800 word	Lưu ảnh sau khi giải mã AES-128.



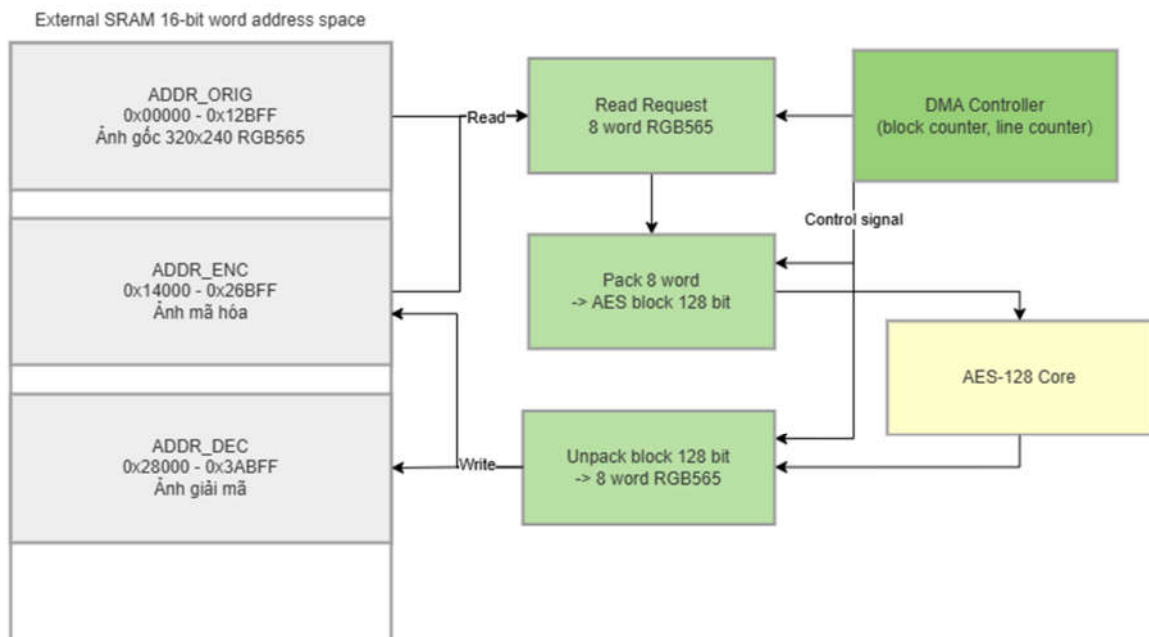
Hình 13. Sơ đồ phân vùng SRAM cho ảnh gốc, ảnh mã hóa và ảnh giải mã

2.8 DMA trong hệ thống xử lý ảnh trên FPGA

Direct Memory Access là cơ chế cho phép một khối phần cứng chuyên dụng đọc và ghi dữ liệu từ bộ nhớ mà không cần CPU điều khiển từng thao tác. Trong hệ thống đề xuất, DMA là cầu nối giữa SRAM và lõi AES. DMA đọc các pixel từ SRAM, ghép thành block AES, kích hoạt lõi AES và ghi kết quả trở lại SRAM.

Việc sử dụng DMA giúp hệ thống có cấu trúc rõ ràng hơn so với việc để controller trung tâm điều khiển từng chu kỳ đọc ghi. Controller chỉ cần phát lệnh start, chọn chế độ encrypt hoặc decrypt và theo dõi busy hoặc done. Chi tiết đọc 8 pixel, pack dữ liệu, chờ AES và ghi 8 pixel kết quả được tách vào module DMA.

DMA trong thiết kế cũng hỗ trợ fast mode và slow mode. Fast mode ưu tiên throughput xử lý AES, trong khi slow mode cố tình giảm tốc để VGA có thêm slot đọc SRAM và người dùng có thể quan sát quá trình thay đổi ảnh. Đây là một lựa chọn thiết kế phù hợp với mục tiêu demo trực quan.



Hình 14. Luồng hoạt động của DMA giữa SRAM và AES core

2.9 Chuẩn VGA 640x480 và hiển thị ảnh từ SRAM

VGA là chuẩn hiển thị sử dụng các tín hiệu đồng bộ ngang, đồng bộ dọc và các kênh màu. Với độ phân giải 640x480, bộ tạo timing cần tạo ra vùng hiển thị và vùng blanking

đúng thông số để màn hình nhận được ảnh ổn định. Trong hệ thống, xung 50 MHz được chia đôi để tạo xung 25 MHz gần với pixel clock thường dùng cho VGA 640x480.

Vì ảnh gốc có kích thước 320x240, màn hình VGA 640x480 có thể chia thành bốn vùng bằng nhau. Thiết kế hệ thống đề xuất sử dụng cách bố trí gồm ORIG ở góc trên trái, dashboard ở góc trên phải, ENC ở góc dưới trái và DEC ở góc dưới phải. Bố cục này giúp người dùng quan sát đồng thời dữ liệu đầu vào, dữ liệu đã mã hóa, dữ liệu đã giải mã và trạng thái hệ thống.

Khối VGA không chỉ hiển thị ảnh mà còn hỗ trợ debug. Dashboard có thể hiển thị trạng thái controller, trạng thái DMA, trạng thái AES core, chủ sở hữu SRAM, số packet UART, giá trị CRC và các cờ trạng thái. Nhờ đó, khi demo trên phần cứng, người dùng có thể xác định hệ thống đang chờ ảnh, đang mã hóa, đang giải mã hay đã hoàn thành.

Bảng 11. Bố cục hiển thị VGA 640x480 trong hệ thống đề xuất

Vùng màn hình	Kích thước	Nội dung hiển thị
Góc trên trái	320 x 240	Ảnh gốc ORIG đọc từ vùng SRAM gốc.
Góc trên phải	320 x 240	Dashboard trạng thái hệ thống.
Góc dưới trái	320 x 240	Ảnh mã hóa ENC đọc từ vùng SRAM mã hóa.
Góc dưới phải	320 x 240	Ảnh giải mã DEC đọc từ vùng SRAM giải mã.



Hình 15. Bố cục VGA 640x480 chia bốn vùng

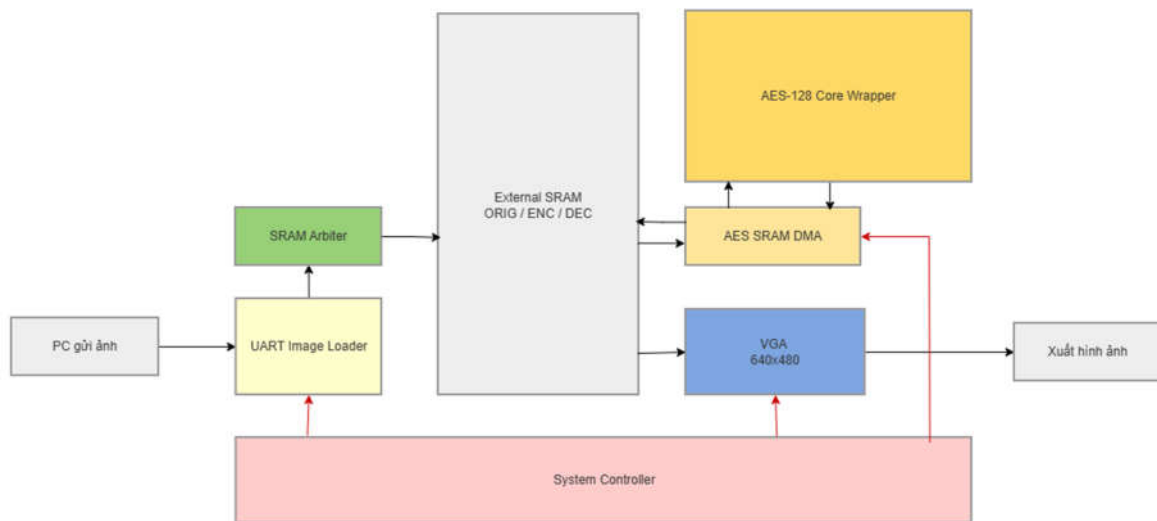
CHƯƠNG 3. PHÂN TÍCH YÊU CẦU VÀ THIẾT KẾ HỆ THỐNG

3.1 Tổng quan bài toán

Bài toán của đề tài là xây dựng một hệ thống phần cứng có khả năng nhận ảnh từ máy tính, lưu ảnh vào SRAM, mã hóa hoặc giải mã ảnh bằng AES-128 và hiển thị kết quả lên màn hình VGA. Hệ thống cần hoạt động trên FPGA, được mô tả bằng Verilog HDL và có thể kiểm thử bằng cả mô phỏng lẫn phần cứng thực tế.

Khác với một thiết kế chỉ kiểm tra AES bằng một block dữ liệu duy nhất, hệ thống đề xuất xử lý một ảnh hoàn chỉnh. Vì vậy, hệ thống cần giải quyết nhiều vấn đề tích hợp như truyền dữ liệu số lượng lớn qua UART, đóng gói packet, kiểm tra lỗi, quản lý địa chỉ SRAM, chia sẻ SRAM giữa nhiều master, điều khiển DMA, đồng bộ với lõi AES và tạo giao diện VGA.

Luồng vận hành chính bắt đầu khi người dùng nạp ảnh từ PC. Phần mềm trên PC chuyển ảnh thành dữ liệu RGB565, chia thành 600 packet và gửi xuống FPGA. FPGA ghi dữ liệu vào vùng ORIG của SRAM. Sau khi ảnh sẵn sàng, người dùng nhấn start để chạy mã hóa hoặc giải mã. Kết quả được lưu vào vùng ENC hoặc DEC và được VGA hiển thị trực quan.



Hình 16. Luồng xử lý tổng quát của hệ thống đề xuất

3.2 Yêu cầu chức năng và phi chức năng

Yêu cầu chức năng của hệ thống tập trung vào việc bảo đảm luồng dữ liệu chính hoạt động đầy đủ. Hệ thống phải nhận được ảnh từ máy tính, ghi đúng ảnh vào SRAM, xử lý

mã hóa và giải mã bằng AES-128, hiển thị được kết quả lên VGA và cung cấp tín hiệu debug để kiểm tra trạng thái vận hành.

Yêu cầu phi chức năng tập trung vào tính ổn định, khả năng mô phỏng, khả năng tổng hợp và khả năng bảo trì mã nguồn. Các khối chức năng cần được tách rõ trách nhiệm, có testbench tự kiểm tra và có mô tả đủ để người đọc hiểu được cách hệ thống vận hành. Đối với báo cáo, mỗi yêu cầu cần có tiêu chí kiểm chứng để tránh trình bày chung chung.

Bảng 12. Bảng phân tích yêu cầu hệ thống

Nội dung yêu cầu	Tiêu chí kiểm chứng
Hệ thống nhận ảnh 320x240 RGB565 từ PC qua UART.	Packet count đạt 600 và cờ cờ ảnh đã nạp bật.
Hệ thống kiểm tra packet bằng CRC8 và phản hồi ACK/NACK.	Packet hợp lệ nhận ACK, packet sai nhận NACK trong testbench.
Hệ thống lưu ảnh gốc, ảnh mã hóa và ảnh giải mã trong SRAM.	Địa chỉ ghi thuộc đúng vùng ORIG, ENC hoặc DEC.
Hệ thống mã hóa ảnh bằng AES-128.	Vùng ENC được ghi sau khi DMA chạy chế độ encrypt.
Hệ thống giải mã ảnh bằng AES-128.	Vùng DEC được ghi sau khi DMA chạy chế độ decrypt.
Hệ thống hiển thị ORIG, ENC, DEC và dashboard qua VGA.	VGA tạo đúng timing và renderer chọn đúng vùng hiển thị.
Thiết kế có cấu trúc module rõ ràng, dễ kiểm thử.	Thiết kế tách nhóm thành phần kỹ thuật, nhóm thành phần kỹ thuật, nhóm thành phần kỹ thuật và có testbench riêng.
Thiết kế tổng hợp được trên Cyclone II.	Quartus flow thành công và có báo cáo tài nguyên.
Các module chính có self-checking testbench.	Danh sách testbench trong nhóm thành phần kỹ thuậtb chạy theo script.

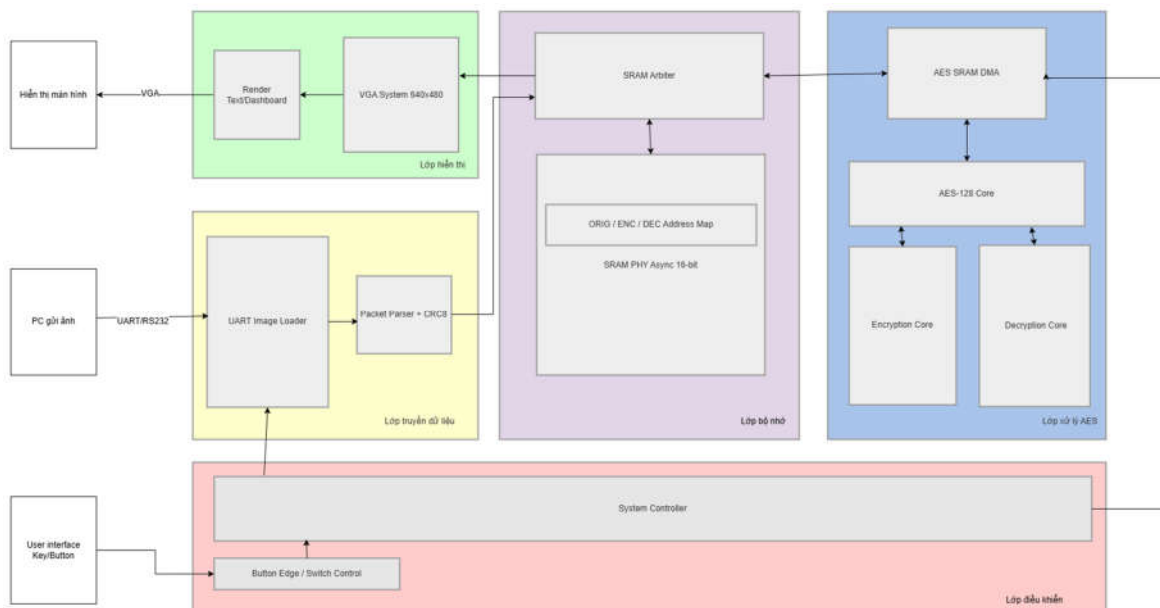
3.3 Kiến trúc tổng quan của hệ thống

Kiến trúc tổng quan của hệ thống được tổ chức theo hướng phân cấp. Lớp ngoài cùng kết nối các chân vật lý của board với khối điều khiển trung tâm. Bên trong, hệ thống tích hợp khối nạp ảnh qua UART, khối DMA xử lý ảnh, bộ phân xử SRAM, khối giao tiếp SRAM, khối AES-128 và khối hiển thị VGA. Cách tổ chức này giúp mỗi thành phần đảm nhiệm một vai trò rõ ràng và thuận tiện cho kiểm thử từng phần.

Cách tách top-level và controller giúp phân cứng dễ quản lý hơn. khối kết nối phần cứng mức cao đóng vai trò board-facing top, còn controller đóng vai trò system top logic.

Nhờ đó, khi thay đổi pin, board hoặc cách hiển thị, người thiết kế có thể giới hạn phạm vi chỉnh sửa mà không làm thay đổi toàn bộ kiến trúc.

Trong kiến trúc hiện tại, các master truy cập SRAM gồm UART loader, AES DMA và VGA reader. Tất cả master không điều khiển trực tiếp chân SRAM mà đi qua khối phân xử truy cập SRAM và khối giao tiếp vật lý SRAM 16 bit. Điều này giúp tránh xung đột bus và tạo một điểm kiểm soát rõ ràng cho chính sách ưu tiên truy cập bộ nhớ.



Hình 17. Sơ đồ khối tổng thể của hệ thống đề xuất

Bảng 13. Các khối chính trong kiến trúc hệ thống đề xuất

Khối	Module đại diện	Chức năng chính
Top phần cứng	khối kết nối phần cứng mức cao	Kết nối chân CLOCK, KEY, SW, UART, SRAM, VGA và LED.
Controller hệ thống	khối điều khiển trung tâm	Điều phối trạng thái load ảnh, idle, encrypt, decrypt và done.
UART loader	khối nạp ảnh qua UART	Nhận packet UART, kiểm tra CRC8 và ghi ảnh gốc vào SRAM.
SRAM arbiter	khối phân xử truy cập SRAM	Phân quyền truy cập SRAM giữa UART, DMA và VGA.
SRAM PHY	khối giao tiếp vật lý SRAM 16 bit	Điều khiển bus địa chỉ, dữ liệu và tín hiệu đọc ghi SRAM ngoài.
AES DMA	khối DMA xử lý ảnh qua AES	Đọc SRAM, pack 8 pixel, xử lý AES và ghi kết quả.
AES wrapper	khối chuẩn hóa giao diện lõi AES-128	Chuẩn hóa lõi AES thành giao diện start, busy và done.

Khối	Module đại diện	Chức năng chính
Khối hiển thị VGA	Tạo timing, đọc pixel từ SRAM và hiển thị dashboard.	Xuất dữ liệu màu và tín hiệu đồng bộ ra màn hình VGA.

3.4 Thiết kế cấu trúc thiết kế và module

Thiết kế hệ thống đề xuất được refactor để tách rõ mã nguồn đang sử dụng và mã nguồn tham khảo. Thư mục nhóm thành phần kỹ thuật chứa các module phần cứng chính. Thư mục nhóm thành phần kỹ thuật chứa model SRAM, AES mock, script ModelSim và testbench. Thư mục nhóm thành phần kỹ thuật chứa công cụ chuyển đổi ảnh và gửi ảnh qua UART. Thư mục nhóm thành phần kỹ thuật chứa tài liệu kiến trúc, UART, test plan và ghi chú refactor.

Hai top-level chính được duy trì là khối kết nối phần cứng mức cao và khối mô phỏng mức cao. khối kết nối phần cứng mức cao dùng cho demo phần cứng với ảnh nạp qua UART, còn khối mô phỏng mức cao dùng cho mô phỏng hoặc smoke test với ảnh từ ROM. Các thư mục nhóm thành phần kỹ thuật và nhóm thành phần kỹ thuật_uart được giữ để tham khảo lịch sử nhưng không thuộc đường build chính.

```

└─ rtl
  └─ aes_core
    ├── AddRoundKey.v
    ├── aes_decryption_core.v
    ├── aes_encryption_core.v
    ├── aes128_core_wrapper.v
    ├── Inv_Mix_Single_Column.v
    ├── Inv_MixColumns.v
    ├── Inv_Sbox_8bits.v
    ├── InvKeyExpansion.v
    ├── InvShiftRows.v
    ├── InvSubBytes.v
    ├── KeyExpansion.v
    ├── Mix_Single_Column.v
    ├── MixColumns.v
    ├── rcon_gen.v
    ├── Sbox_8bits.v
    ├── ShiftRows.v
    └── SubBytes.v
  └─ control
    ├── aes_image_demo_controller_uart.v
    ├── aes_image_demo_controller.v
    ├── button_edge.v
    └── input_control.v
  └─ dma
    ├── aes_sram_dma_320x240.v
    └── image_loader_320x240.v
  └─ rom
    └── image_rom_320x240_rgb565.v
  └─ sram
    ├── sram_arbiter_3m.v
    ├── sram_arbiter.v
    └── sram_phy_async16.v
  └─ uart
    ├── baud_rate_gen.v
    ├── uart_controller.v
    ├── uart_image_loader_320x240.v
    ├── uart_rx_packet_256.v
    ├── uart_rx.v
    ├── uart_sram_packet_writer_320x240.v
    └── uart_tx.v
  └─ vga
    ├── font_rom_8x8.v
    ├── text_dashboard.v
    ├── vga_quadrant_renderer_320x240.v
    ├── vga_sram_reader.v
    ├── vga_system_640x480.v
    ├── vga_timing_640x480.v
    ├── top_de.v
    └── top.v
  └─ sim
    ├── models
    ├── scripts
    └─ tb
      ├── tb_aes_sram_dma_320x240_selftest.v
      ├── tb_aes128_core_wrapper_selftest.v
      ├── tb_image_loader_selftest.v
      ├── tb_sram_arbiter_selftest.v
      ├── tb_system_smoke_small.v
      ├── tb_uart_rx_packet_256_selftest.v
      ├── tb_uart_sram_packet_writer_selftest.v
      └── tb_vga_timing_waveform.v
  └─ tools
```

Hình 18. Cấu trúc thư mục thiết kế hệ thống đề xuất**Bảng 14. Bảng mô tả các thư mục chính trong thiết kế**

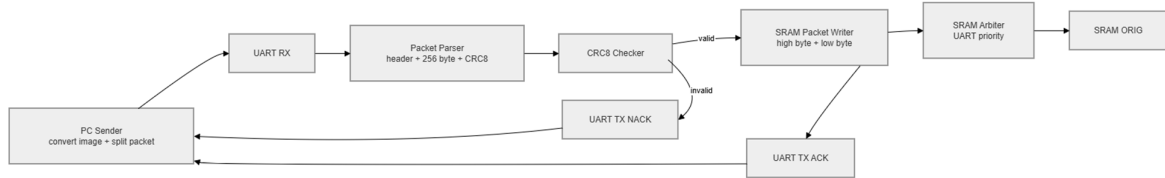
Thư mục	Vai trò	Ghi chú
nhóm thành phần kỹ thuật	Chứa mã nguồn Verilog RTL đang được duy trì.	Là phần chính để tổng hợp và mô phỏng.
nhóm thành phần kỹ thuật	Chứa model, script và testbench.	Dùng cho ModelSim-Altera.
nhóm thành phần kỹ thuật	Chứa script Python chuyển ảnh và gửi ảnh UART.	Có GUI và CLI cho gửi ảnh.
nhóm thành phần kỹ thuật	Chứa project Quartus chính.	Entity chính là khối hệ thống chính.
nhóm thành phần kỹ thuật	Chứa tài liệu kiến trúc và test plan.	Dùng làm cơ sở viết báo cáo.
Nhóm ảnh kết quả	Chứa ảnh minh họa trạng thái nạp ảnh, mã hóa và giải mã.	Dùng để thay vào các placeholder trong báo cáo.

3.5 Thiết kế khối giao tiếp UART và nạp ảnh

Khối UART là đầu vào chính của hệ thống phần cứng thật. Trong phiên bản khối kết nối phần cứng mức cao, FPGA chờ máy tính gửi ảnh trước khi cho phép người dùng chạy AES. Điều này giúp demo linh hoạt hơn so với việc nhúng ảnh cố định vào ROM nội bộ, vì người dùng có thể thay đổi ảnh mà không cần rebuild Quartus.

Luồng nạp ảnh gồm ba lớp. Lớp đầu tiên là khối nhận UART và khối phát UART, chịu trách nhiệm nhận dữ liệu ảnh từ máy tính và gửi phản hồi ACK hoặc NACK. Lớp thứ hai là khối nhận và kiểm tra packet UART, chịu trách nhiệm gom byte thành packet, kiểm tra header và CRC8, sau đó tạo packet hợp lệ. Lớp thứ ba là khối ghi dữ liệu packet vào SRAM, chịu trách nhiệm ghi payload vào SRAM theo địa chỉ tương ứng.

Module khối nạp ảnh qua UART đóng vai trò tích hợp các khối trên. Module này nhận byte UART, kiểm tra packet, ghi dữ liệu ảnh gốc vào SRAM và xuất các tín hiệu debug như số packet đã nhận, CRC tính được, CRC nhận được, trạng thái packet và trạng thái ghi. Khi nhận đủ 600 packet, module bật cờ chờ ảnh đã nạp để controller chuyển sang trạng thái sẵn sàng.



Hình 19. Sơ đồ luồng nạp ảnh qua UART vào SRAM

Bảng 15. Các module trong khối UART image loader

Module	Input chính	Output chính	Chức năng
Khởi tạo baudrate	clk, reset	tick lấy mẫu UART	Tạo xung lấy mẫu theo baudrate.
Khởi nhận UART	đường nhận UART, tick	dữ liệu nhận, cờ nhận xong	Nhận một byte UART.
Khởi phát UART	dữ liệu phát, tín hiệu bắt đầu phát	đường phát UART, trạng thái bận	Gửi ACK hoặc NACK về máy tính.
khởi nhận và kiểm tra packet UART	dữ liệu nhận, nhận xong	packet hợp lệ, CRC debug	Gom packet 256 byte và kiểm tra CRC8.
khởi ghi dữ liệu packet vào SRAM	packet hợp lệ, packet data	yêu cầu truy cập SRAM, địa chỉ SRAM, dữ liệu ghi SRAM	Ghi payload packet vào SRAM.
khởi nạp ảnh qua UART	UART_RXD, enable	cờ ảnh đã nạp, UART_TXD	Tích hợp toàn bộ luồng nạp ảnh.

Địa chỉ ghi SRAM được tính dựa trên chỉ số packet và chỉ số word trong packet. Mỗi packet có 256 byte, tương đương 128 word RGB565. Vì ảnh có 600 packet, bộ đếm packet cần đủ để biểu diễn từ 0 đến 599. Khi packet hợp lệ, writer ghi tuần tự 128 word vào SRAM. Khi packet lỗi, hệ thống gửi NACK và không tăng số packet đã nhận.

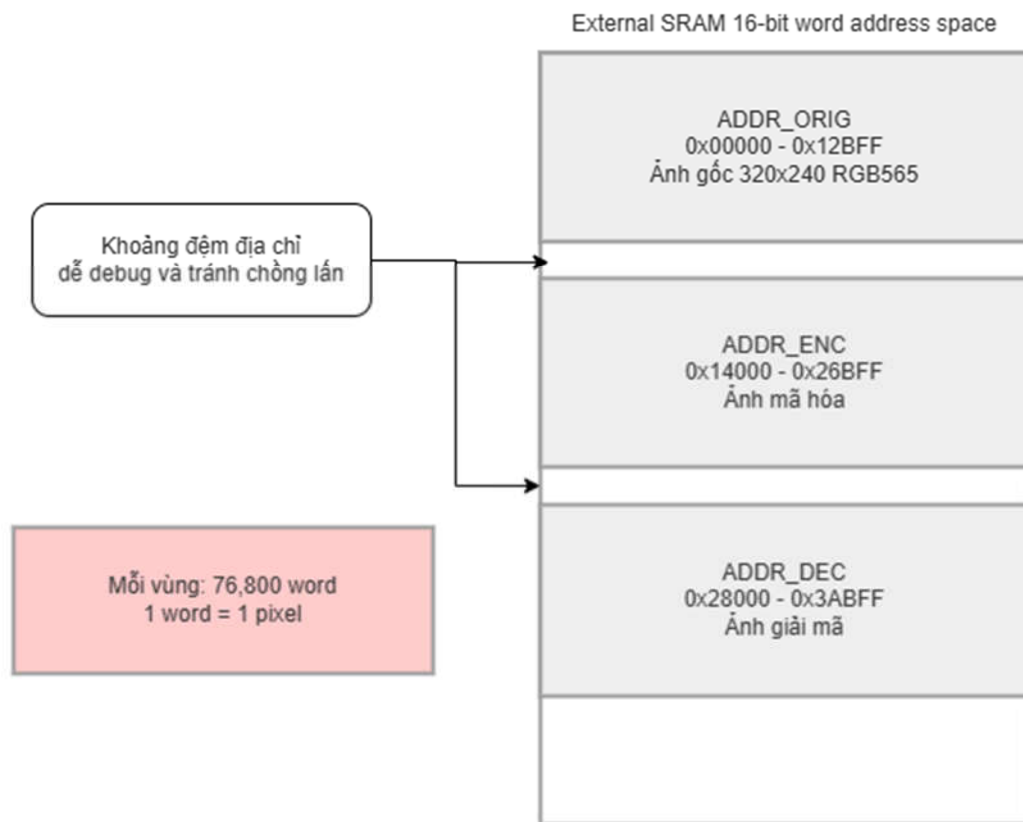
Cách tổ chức này giúp hệ thống có khả năng kiểm soát lỗi ở mức packet. Nếu một packet sai CRC, dữ liệu của packet đó không được ghi vào SRAM và phần mềm PC có thể gửi lại. Điều này quan trọng vì một ảnh có 153.600 byte, chỉ cần một lỗi truyền cũng có thể làm lệch dữ liệu ảnh hoặc làm sai kết quả AES.

3.6 Thiết kế tổ chức bộ nhớ SRAM

SRAM được tổ chức theo ba vùng nhớ độc lập để lưu ảnh gốc, ảnh mã hóa và ảnh giải mã. Cách phân vùng này có hai lợi ích chính. Thứ nhất, dữ liệu ảnh gốc không bị ghi đè khi chạy mã hóa. Thứ hai, người dùng có thể hiển thị đồng thời cả ba ảnh để kiểm tra quá trình xử lý.

Mỗi vùng ảnh có kích thước 76.800 word 16 bit. Địa chỉ bắt đầu của ba vùng lần lượt là 18'h00000, 18'h14000 và 18'h28000. Khoảng cách giữa các địa chỉ bắt đầu lớn hơn kích thước vùng ảnh, tạo ra vùng trống để giảm nguy cơ chồng lấn và giúp việc debug địa chỉ dễ hơn.

Trong báo cáo chính thức, nên chèn sơ đồ bộ nhớ thể hiện rõ khoảng địa chỉ của từng vùng. Sơ đồ này đặc biệt quan trọng vì nhiều lỗi hệ thống xử lý ảnh trên FPGA xuất phát từ việc tính địa chỉ sai, đảo thứ tự dòng hoặc ghi đè vùng dữ liệu.



Hình 20. Address map SRAM của hệ thống

Bảng 16. Tính toán dung lượng SRAM cho ảnh 320x240 RGB565

Đại lượng	Công thức	Kết quả
Số pixel một ảnh	320 x 240	76.800 pixel
Số word một ảnh	76.800 x 1 word	76.800 word
Số byte một ảnh	76.800 x 2 byte	153.600 byte
Số word ba vùng ảnh	76.800 x 3	230.400 word
Số byte ba vùng ảnh	230.400 x 2	460.800 byte

3.7 Thiết kế khối phân xử truy cập SRAM

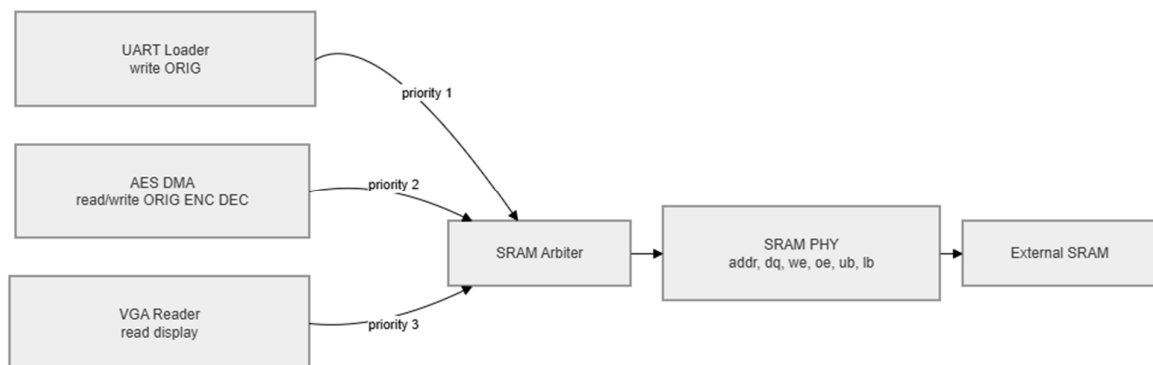
SRAM ngoài là tài nguyên dùng chung nên không thể để nhiều module điều khiển bus cùng lúc. Trong hệ thống đề xuất, có ba master cần truy cập SRAM là UART loader, AES DMA và VGA reader. Nếu không có arbiter, các tín hiệu địa chỉ, dữ liệu và điều khiển có thể xung đột, làm sai dữ liệu hoặc gây trạng thái không xác định trên bus.

Module khối phân xử truy cập SRAM giải quyết vấn đề này bằng cách cấp quyền theo thứ tự ưu tiên. UART loader có ưu tiên cao nhất, AES DMA có ưu tiên thứ hai và VGA reader có ưu tiên thấp nhất. Chính sách này phù hợp với yêu cầu hệ thống vì trong giai đoạn nạp ảnh, UART cần ghi đủ dữ liệu để tránh mất packet. Trong giai đoạn AES fast mode, DMA cần ưu tiên để đảm bảo throughput. VGA có thể đọc khi hệ thống cho phép, đặc biệt trong slow mode.

Arbiter xuất tín hiệu chủ truy cập debug để dashboard VGA hiển thị master đang giữ bus. Đây là một điểm hỗ trợ debug quan trọng. Khi ảnh không hiển thị hoặc DMA không chạy, người dùng có thể quan sát owner để biết SRAM đang được UART, DMA hay VGA sử dụng.

Bảng 17. Chính sách ưu tiên truy cập SRAM

Mức ưu tiên	Master	Lý do ưu tiên
1	UART loader	Cần ghi packet kịp thời khi đang nạp ảnh từ PC.
2	AES DMA	Cần đọc ghi dữ liệu khối khi đang mã hóa hoặc giải mã.
3	VGA reader	Có thể bị tạm dừng khi UART hoặc DMA cần throughput.



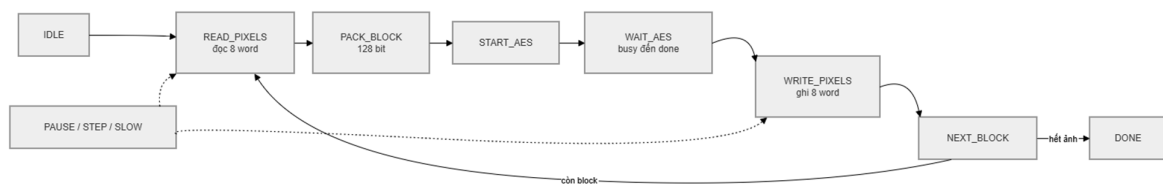
Hình 21. Sơ đồ phân xử SRAM giữa UART, DMA và VGA

3.8 Thiết kế DMA xử lý dữ liệu ảnh

DMA là thành phần trung tâm của quá trình xử lý ảnh bằng AES. Module này nhận lệnh start từ controller, nhận chế độ decrypt hoặc encrypt, sau đó tự điều khiển quá trình đọc dữ liệu từ SRAM, đưa dữ liệu vào AES và ghi kết quả trở lại SRAM. Nhờ đó, controller không cần xử lý từng pixel mà chỉ quản lý trạng thái tổng thể.

Một block AES gồm 128 bit. Vì một pixel RGB565 có 16 bit, DMA cần đọc 8 word từ SRAM để tạo một block AES. Khi đọc đủ 8 word, DMA phát tín hiệu bắt đầu AES, chờ tín hiệu AES hoàn tất, sau đó tách aes block dữ liệu ra thành 8 word và ghi vào vùng đích. Nếu đang mã hóa, nguồn là ORIG và đích là ENC. Nếu đang giải mã, nguồn là ENC và đích là DEC.

DMA còn xuất các tín hiệu debug như state, bộ đếm block, bộ đếm dòng, bộ đếm block theo cột, pixel index, bộ đếm chu kỳ và bộ đếm chu kỳ hoạt động. Những tín hiệu này được dùng để hiển thị lên dashboard hoặc kiểm tra waveform trong mô phỏng. Việc đưa debug ra ngoài giúp quá trình kiểm thử dễ hơn nhiều so với việc chỉ quan sát done cuối cùng.



Hình 22. FSM đề xuất của AES SRAM DMA

Bảng 18. Nguồn và đích dữ liệu theo chế độ DMA

Chế độ	Vùng nguồn	Vùng đích	Mục đích
Encrypt	ORIG	ENC	Tạo ảnh mã hóa từ ảnh gốc.
Decrypt	ENC	DEC	Khôi phục ảnh giải mã từ ảnh đã mã hóa.
Auto	ORIG rồi ENC	ENC rồi DEC	Mã hóa xong tự chạy giải mã.

Với ảnh 320x240, DMA xử lý 9.600 block. Nếu chỉ xét số lần đọc và ghi SRAM, mỗi block cần 8 lần đọc nguồn và 8 lần ghi đích. Ngoài ra còn có thời gian chờ AES hoàn thành. Vì vậy, tổng thời gian xử lý phụ thuộc vào độ trễ lỗi AES, tốc độ SRAM, chính sách arbiter và chế độ fast hoặc slow.

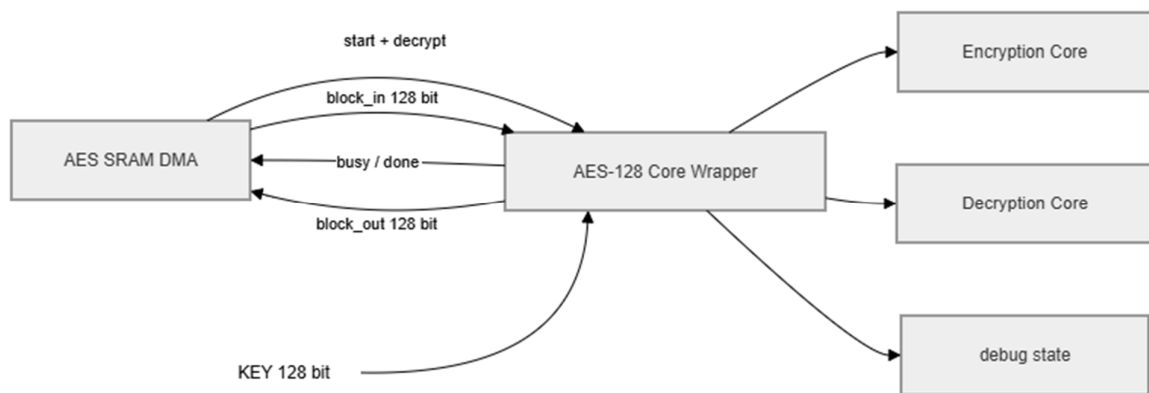
Trong fast mode, hệ thống ưu tiên DMA và có thể tạm tắt đọc ảnh của VGA để không ảnh hưởng throughput. Trong slow mode, DMA được throttle theo frame tick hoặc mức slow để VGA có cơ hội đọc SRAM. Điều này làm giảm tốc độ xử lý nhưng cải thiện khả năng quan sát trực quan khi demo.

3.9 Thiết kế AES-128 core wrapper

Lỗi AES được tích hợp vào hệ thống thông qua một lớp giao diện điều khiển nhằm chuẩn hóa cách kích hoạt và cách báo trạng thái. Lớp giao diện này chuyển quá trình xử lý AES thành một cơ chế handshake gồm tín hiệu bắt đầu, tín hiệu đang bận và tín hiệu hoàn tất. Đây là quyết định thiết kế quan trọng vì các khối điều khiển hệ thống thường dễ phối hợp hơn khi dùng giao diện đồng nhất và có trạng thái rõ ràng.

Wrapper nhận tín hiệu decrypt để quyết định kích hoạt lõi mã hóa hoặc lõi giải mã. Khi start được phát, wrapper chốt chế độ xử lý, kéo tín hiệu bắt đầu mức thấp tương ứng xuống một chu kỳ, chuyển sang trạng thái chờ và theo dõi done của lõi bên trong. Khi lõi hoàn thành, wrapper phát done một chu kỳ và trở về idle.

Cách bọc này giúp giữ nguyên AES core gốc nhưng vẫn làm cho lõi AES trở thành một IP dễ tích hợp. DMA không cần biết lõi bên trong dùng tín hiệu bắt đầu mức thấp hay bao nhiêu trạng thái nội bộ. DMA chỉ cần tuân thủ giao thức đơn giản là phát start khi AES không busy và chờ done trước khi đọc block dữ liệu ra.



Hình 23. Giao diện chuẩn hóa của khối giao diện lõi AES

Bảng 19. Tín hiệu chính của AES wrapper

Tín hiệu	Hướng	Mô tả
start	Input	Yêu cầu bắt đầu xử lý một block 128 bit.
decrypt	Input	0 là mã hóa, 1 là giải mã.

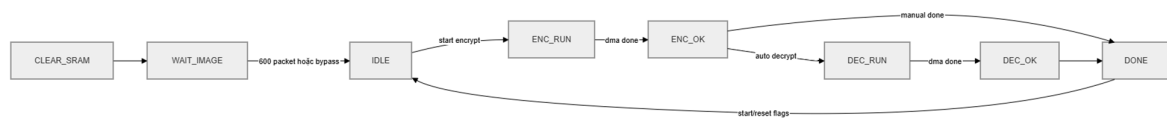
Tín hiệu	Hướng	Mô tả
block dữ liệu vào[127:0]	Input	Block dữ liệu đầu vào từ DMA.
key[127:0]	Input	Khóa AES-128.
block dữ liệu ra[127:0]	Output	Block kết quả sau mã hóa hoặc giải mã.
busy	Output	AES đang xử lý.
done	Output	AES hoàn thành một block.
trạng thái lỗi xử lý_dbg[3:0]	Output	Trạng thái debug của wrapper.

3.10 Thiết kế controller hệ thống

Khởi điều khiển trung tâm quyết định trình tự vận hành của toàn bộ hệ thống. Sau reset, hệ thống có thể thực hiện giai đoạn khởi tạo bộ nhớ, sau đó chuyển sang chờ ảnh từ UART. Khi ảnh đã được nạp hoặc người dùng chọn chế độ bỏ qua bước chờ, controller cho phép hệ thống vào trạng thái chờ lệnh. Từ trạng thái này, người dùng có thể kích hoạt mã hóa, giải mã hoặc chế độ tự động mã hóa rồi giải mã.

Các trạng thái chính của controller gồm C_CLEAR_SRAM, C_WAIT_IMAGE, C_IDLE, C_ENC_RUN, C_ENC_OK, C_DEC_RUN, C_DEC_OK và C_DONE. Trạng thái C_WAIT_IMAGE bảo đảm AES không chạy khi dữ liệu gốc chưa sẵn sàng. Trạng thái C_ENC_RUN và C_DEC_RUN giữ hệ thống trong quá trình DMA hoạt động. Khi DMA done, controller cập nhật cờ hoàn thành và chuyển sang trạng thái tiếp theo.

Các nút nhấn và công tắc được xử lý thông qua một khối nhập điều khiển riêng để tạo các xung start, pause, step và các tín hiệu chọn chế độ. Cách tách này giúp controller không phải trực tiếp xử lý chống dội phím hoặc giải mã trạng thái công tắc, từ đó giữ cho FSM điều khiển trung tâm dễ đọc và dễ kiểm thử hơn.



Hình 24. FSM điều khiển tổng thể của hệ thống

Bảng 20. Bảng trạng thái controller hệ thống

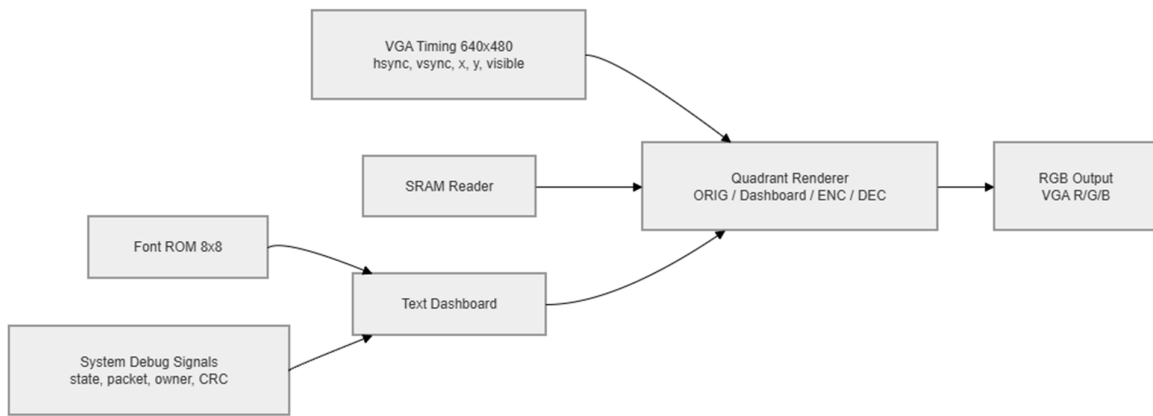
Trạng thái	Ý nghĩa	Điều kiện chuyển trạng thái
C_CLEAR_SRAM	Xóa hoặc khởi tạo vùng hiển thị SRAM.	Hoàn thành clear thì chuyển WAIT_IMAGE.
C_WAIT_IMAGE	Chờ ảnh nạp từ UART hoặc bypass.	cờ ảnh sẵn sàng bằng 1.
C_IDLE	Sẵn sàng nhận lệnh từ người dùng.	Nhấn start để chạy encrypt hoặc decrypt.
C_ENC_RUN	DMA đang mã hóa từ ORIG sang ENC.	DMA hoàn tất bằng 1.
C_ENC_OK	Mã hóa hoàn thành.	Auto mode thì chạy decrypt, không thì DONE.
C_DEC_RUN	DMA đang giải mã từ ENC sang DEC.	DMA hoàn tất bằng 1.
C_DEC_OK	Giải mã hoàn thành.	Chuyển DONE.
C_DONE	Hoàn thành thao tác hiện tại.	Người dùng reset hoặc start lại tùy cấu hình.

3.11 Thiết kế hiển thị VGA và dashboard debug

Khối VGA được thiết kế thành ba lớp chính. Lớp thứ nhất tạo tọa độ điểm ảnh, tín hiệu hiển thị hợp lệ, HSYNC và VSYNC. Lớp thứ hai quyết định địa chỉ SRAM cần đọc dựa trên tọa độ hiện tại và vùng hiển thị. Lớp thứ ba chuyển dữ liệu pixel RGB565 và thông tin dashboard thành tín hiệu màu VGA. Cách phân lớp này giúp tách rõ timing, truy cập bộ nhớ và dựng hình.

Dashboard là thành phần hữu ích trong demo phần cứng vì giúp người dùng quan sát trạng thái mà không cần logic analyzer. Dashboard có thể hiển thị trạng thái hệ thống, trạng thái DMA, trạng thái lỗi AES, chủ truy cập SRAM, chế độ nhanh hoặc chậm, chế độ mã hóa hoặc giải mã, tiến độ xử lý block, tiến độ nạp ảnh và thông tin kiểm tra CRC.

Tín hiệu cho phép xem bus được dùng để điều khiển việc VGA có được đọc ảnh từ SRAM hay không. Khi UART đang nạp ảnh hoặc DMA đang chạy fast mode, hệ thống có thể tạm tắt VGA image read để tránh ảnh hưởng đến throughput của UART hoặc DMA. Khi slow mode, VGA có cơ hội đọc nhiều hơn, giúp người dùng quan sát quá trình ảnh thay đổi.



Hình 25. Kiến trúc VGA subsystem và dashboard

Bảng 21. Các thông tin debug nên hiển thị trên dashboard

Nhóm thông tin	Tín hiệu đại diện	Ý nghĩa
Trạng thái hệ thống	trạng thái hệ thống, trạng thái DMA, trạng thái lỗi xử lý	Cho biết hệ thống đang chờ ảnh, đang mã hóa hoặc đang giải mã.
Trạng thái bus	chủ truy cập bus	Cho biết UART, DMA hay VGA đang truy cập SRAM.
Chế độ chạy	chế độ nhanh, mức chậm, chế độ giải mã, chế độ tự động	Cho biết cách người dùng cấu hình hệ thống.
Tiến độ DMA	bộ đếm block, bộ đếm dòng, bộ đếm block theo cột	Cho biết vị trí xử lý hiện tại trong ảnh.
UART debug	số packet đã nhận, CRC tính được, CRC nhận được, cờ UART	Cho biết tiến độ nạp ảnh và kết quả kiểm tra packet.

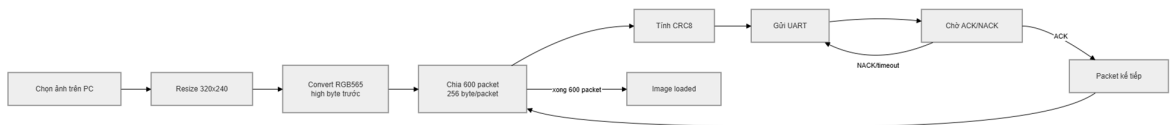
3.12 Thiết kế phần mềm gửi ảnh từ máy tính

Phần mềm phía máy tính có nhiệm vụ chuẩn bị dữ liệu ảnh cho FPGA. Script công cụ gửi ảnh.py hỗ trợ cả chế độ GUI và CLI. Ở chế độ GUI, người dùng có thể chọn ảnh và cổng COM thuận tiện hơn. Ở chế độ CLI, người dùng truyền đường dẫn ảnh và port UART qua tham số dòng lệnh.

Trước khi gửi, ảnh được chuyển về kích thước 320x240 và mã hóa màu sang RGB565. Sau đó, dữ liệu được chia thành 600 packet, mỗi packet chứa 256 byte. Script gửi từng packet, chờ ACK hoặc NACK từ FPGA và xử lý truyền lại nếu cần. Cách làm này giúp tách phần xử lý ảnh trên máy tính khỏi phần xử lý phần cứng trên FPGA.

Ngoài chương trình gửi ảnh qua UART, hệ thống có thể sử dụng công cụ chuyển đổi ảnh sang định dạng RGB565 để phục vụ mô phỏng hoặc nạp trước dữ liệu kiểm thử. Cách hỗ trợ cả luồng nạp trực tiếp và luồng dữ liệu chuẩn bị trước giúp quá trình phát triển linh hoạt hơn, đặc biệt khi cần kiểm tra nhanh phần AES hoặc phần hiển thị mà không phụ thuộc vào truyền UART.

```
python nhóm thành phần kỹ thuật công cụ gửi ảnh.py --gui
python nhóm thành phần kỹ thuật công cụ gửi ảnh.py --path nhóm thành phần kỹ thuật dữ
liệu minh họa --port COM3
python nhóm thành phần kỹ thuật convert_image_to_rgb565_hex.py nhóm thành phần
kỹ thuật dữ liệu minh họa --out-prefix dữ liệu ảnh RGB565
```



Hình 26. Luồng xử lý của phần mềm gửi ảnh trên PC

3.13 Tổng kết thiết kế hệ thống

Thiết kế hệ thống đề xuất được xây dựng theo hướng rõ luồng dữ liệu và rõ trách nhiệm module. UART loader chịu trách nhiệm đưa dữ liệu vào hệ thống, SRAM đóng vai trò bộ nhớ trung tâm, DMA đảm nhiệm việc vận chuyển block ảnh qua AES, AES wrapper xử lý mã hóa hoặc giải mã, VGA subsystem hiển thị kết quả và controller điều phối trình tự vận hành.

Điểm quan trọng của thiết kế là hệ thống không chỉ hiện thực thuật toán AES riêng lẻ mà còn tích hợp thuật toán vào một luồng xử lý ảnh có thể quan sát trực quan. Điều này làm cho đề tài có giá trị thực hành cao vì người thực hiện phải xử lý đồng thời các vấn đề thường gặp trong thiết kế số như handshake, arbitration, memory map, testbench và timing.

CHƯƠNG 4. THỰC HIỆN, KIỂM THỬ VÀ ĐÁNH GIÁ KẾT QUẢ

4.1 Môi trường thực hiện

Quá trình thực hiện và đánh giá hệ thống được đặt trong môi trường phát triển FPGA dùng Quartus II và ModelSim-Altera. Quartus II được sử dụng cho phân tích, tổng hợp, ánh xạ tài nguyên và kiểm tra timing. ModelSim-Altera được sử dụng để chạy testbench, quan sát waveform và kiểm chứng hành vi của các khối trước khi triển khai lên phần cứng.

Thiết kế chính hướng đến FPGA Cyclone II EP2C35F672C6 và sử dụng clock hệ thống 50 MHz. Quartus II 13.0.1 Web Edition được sử dụng để phân tích, tổng hợp, fitter, assembler và TimeQuest Timing Analyzer. Kết quả flow cho biết thiết kế tổng hợp thành công và tạo được cấu hình FPGA, qua đó chứng minh hệ thống có thể triển khai trên phần cứng mục tiêu trong phạm vi kiểm thử hiện tại.

Đối với mô phỏng, các testbench được đặt trong nhóm thành phần kỹ thuật và có thể chạy thông qua script ModelSim. Các testbench được thiết kế theo hướng self-checking, nghĩa là testbench tự so sánh kết quả với giá trị mong đợi và in trạng thái pass hoặc fail. Cách kiểm thử này phù hợp với báo cáo kỹ thuật vì kết quả có thể được ghi lại bằng log và waveform.

Bảng 22. Môi trường thực hiện và kiểm thử

Thành phần	Cấu hình hoặc phiên bản	Vai trò
Ngôn ngữ thiết kế	Verilog HDL	Mô tả module RTL và testbench.
Công cụ tổng hợp	Quartus II 13.0.1 Web Edition	Phân tích, tổng hợp và tạo cấu hình SOF.
Công cụ mô phỏng	ModelSim-Altera	Chạy testbench và quan sát waveform.
FPGA target	Cyclone II EP2C35F672C6	Thiết bị mục tiêu theo báo cáo Quartus.
Ảnh test	test0, test1, test2	Dữ liệu ảnh dùng để demo load, encrypt và decrypt.
PC tool	Python, Pillow, pyserial	Chuyển ảnh và gửi packet qua UART.

4.2 Tổ chức mã nguồn thực hiện

Mã nguồn RTL được chia theo nhóm chức năng. Nhóm lõi AES chứa lõi AES gốc và wrapper. Nhóm control chứa controller hệ thống và xử lý input KEY/SW. Nhóm dma chứa DMA và loader ảnh từ ROM. Nhóm sram chứa arbiter và PHY. Nhóm uart chứa các

module truyền nhận UART và packet writer. Nhóm vga chứa timing, reader, renderer và dashboard.

Cách tổ chức này phù hợp với nguyên tắc thiết kế phân cấp trong Verilog HDL. Mỗi module có giao diện rõ ràng và chỉ xử lý một nhóm chức năng. Khi kiểm thử, nhóm có thể chạy testbench theo từng khối trước khi ghép vào hệ thống lớn. Khi tổng hợp, QSF chỉ cần include các source đang thuộc đường thiết kế chính.

Bảng 23. Các nhóm mã nguồn RTL chính

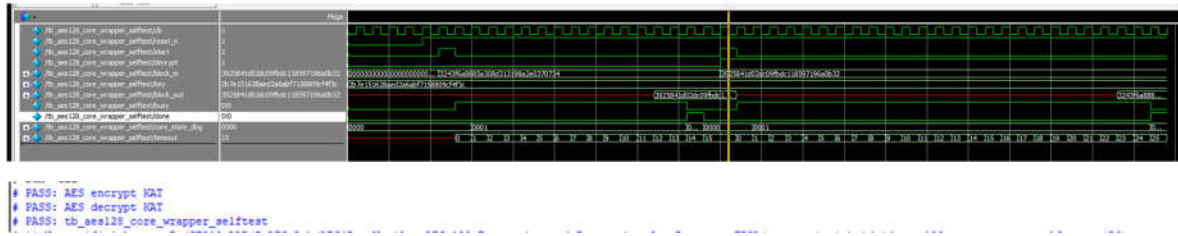
Nhóm RTL	Các module đại diện	Vai trò
nhóm thành phần kỹ thuật lõi AES	khối giao diện lõi AES, lõi mã hóa AES, lõi giải mã AES	Xử lý AES-128 encryption và decryption.
nhóm thành phần kỹ thuật control	khối điều khiển trung tâm, khối nhập điều khiển, xung cạnh nút nhấn	Điều khiển trạng thái hệ thống và input người dùng.
Khối DMA xử lý ảnh	khối DMA xử lý ảnh, khối nạp ảnh	Vận chuyển dữ liệu ảnh giữa SRAM và AES.
Khối bộ nhớ SRAM	khối phân xử SRAM, khối giao tiếp SRAM	Phân xử và điều khiển SRAM ngoài.
Khối nạp dữ liệu UART	khối nạp ảnh qua UART, khối nhận packet UART, khối ghi packet vào SRAM	Khối nạp ảnh và định dạng packet.
Khối hiển thị VGA	hệ thống hiển thị VGA, khối tạo timing VGA, khối đọc SRAM cho VGA, dashboard văn bản	Hiển thị ảnh và trạng thái debug.

4.3 Kiểm thử AES core wrapper

Kiểm thử AES core wrapper cần xác nhận hai nội dung chính. Thứ nhất, wrapper phải kích hoạt đúng lõi encryption hoặc decryption theo tín hiệu decrypt. Thứ hai, kết quả mã hóa và giải mã phải khớp với known-answer vector chuẩn của AES-128. Testbench testbench kiểm thử giao diện lõi AES được dùng cho mục tiêu này.

Known-answer vector thường sử dụng plaintext 3243f6a8885a308d313198a2e0370734, key 2b7e151628aed2a6abf7158809cf4f3c và ciphertext 3925841d02dc09fbdc118597196a0b32 hoặc vector tương ứng trong FIPS 197. Khi encryption cho kết quả đúng và decryption khôi phục lại plaintext, có thể kết luận wrapper và lõi AES hoạt động đúng trong phạm vi test vector.

Khi chèn hình vào báo cáo, nên đưa waveform thể hiện các tín hiệu start, decrypt, busy, done, block dữ liệu vào và block dữ liệu ra. Hình này giúp người đọc thấy rõ giao thức handshake giữa DMA và AES core.



Hình 27. Waveform kiểm thử AES wrapper với known-answer vector

Bảng 24. Testcase kiểm thử AES wrapper

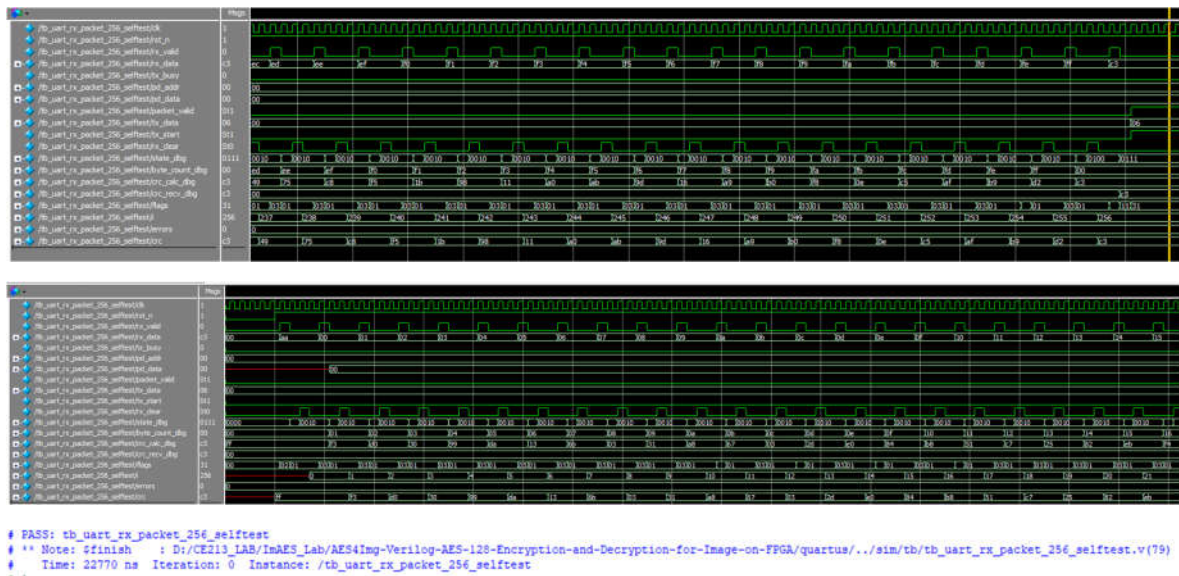
Test ID	Mục tiêu	Input	Expected Output	Kết quả
AES-TC-01	Kiểm tra mã hóa AES-128.	Plaintext và key chuẩn.	Ciphertext khớp vector mong đợi.	Pass theo self-test.
AES-TC-02	Kiểm tra giải mã AES-128.	Ciphertext và key chuẩn.	Plaintext được khôi phục.	Pass theo self-test.
AES-TC-03	Kiểm tra handshake wrapper.	start một chu kỳ.	busy bật trong lúc xử lý và done bật một chu kỳ.	Pass theo waveform.

4.4 Kiểm thử UART packet và CRC8

Khối UART packet được kiểm thử để bảo đảm hệ thống nhận đúng cấu trúc packet và phản hồi chính xác. Testbench testbench kiểm thử nhận packet UART tập trung vào parser packet, trong khi testbench kiểm thử ghi packet vào SRAM kiểm tra quá trình ghi payload vào SRAM. Hai testbench này giúp tách riêng lỗi truyền packet và lỗi ghi bộ nhớ.

Các trường hợp cần kiểm thử gồm packet có header đúng và CRC đúng, packet có CRC sai, packet thiếu byte, packet có header sai và quá trình nhận liên tiếp nhiều packet. Với packet hợp lệ, hệ thống phải tạo packet hợp lệ và trả ACK. Với packet lỗi, hệ thống phải trả NACK và không ghi payload vào SRAM.

Trong báo cáo, nên chèn waveform cho trường hợp một packet đúng và một packet sai. Hình cần làm rõ nhận xong, dữ liệu nhận, packet hợp lệ, CRC tính được, CRC nhận được, dữ liệu phát, bắt đầu phát và phản hồi ACK/NACK. Đây là bằng chứng quan trọng cho thấy hệ thống nạp ảnh có kiểm tra lỗi chứ không chỉ nhận byte thô.



Hình 28. Waveform kiểm thử UART packet đúng CRC và sai CRC

Bảng 25. Testcase kiểm thử UART packet

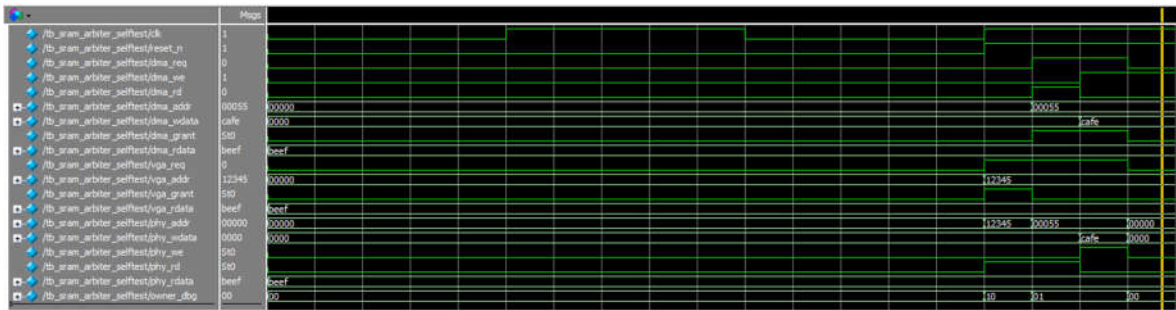
Test ID	Mục tiêu	Input	Expected Output	Kết quả
UART-TC-01	Packet hợp lệ.	Header 0xAA, 256 byte data, CRC đúng.	packet hợp lệ bật và trả ACK.	Pass.
UART-TC-02	Packet sai CRC.	Header đúng nhưng CRC sai.	Không packet hợp lệ và trả NACK.	Pass.
UART-TC-03	Ghi payload vào SRAM.	packet hợp lệ và 256 byte data.	Ghi 128 word vào địa chỉ đúng.	Pass.
UART-TC-04	Đếm packet toàn ảnh.	600 packet hợp lệ.	cờ ảnh đã nạp bật khi đủ packet.	Cần xác nhận bằng phần cứng hoặc log.

4.5 Kiểm thử SRAM arbiter

SRAM arbiter được kiểm thử để xác nhận thứ tự ưu tiên giữa UART, DMA và VGA. Vì đây là điểm giao nhau của nhiều luồng dữ liệu, lỗi arbiter có thể gây sai dữ liệu rất khó phát hiện. Testbench kiểm thử phân xử SRAM cần kiểm tra các trường hợp một master yêu cầu, nhiều master yêu cầu đồng thời và tín hiệu chủ truy cập debug.

Kết quả mong đợi là khối UART luôn được cấp quyền khi có yêu cầu ghi dữ liệu ảnh, khối DMA chỉ được cấp quyền khi UART không yêu cầu và khối VGA chỉ đọc khi hai khối còn lại không cần truy cập. Khi không có yêu cầu nào, trạng thái chủ truy cập phải trở

về rộng. Đây là logic tổ hợp đơn giản nhưng có vai trò rất quan trọng trong toàn bộ hệ thống.



Hình 29. Waveform kiểm thử ưu tiên SRAM arbiter

Bảng 26. Testcase kiểm thử SRAM arbiter

Test ID	UART req	DMA req	VGA req	Master được cấp quyền
ARB-TC-01	1	0	0	UART
ARB-TC-02	0	1	0	DMA
ARB-TC-03	0	0	1	VGA
ARB-TC-04	1	1	1	UART
ARB-TC-05	0	1	1	DMA
ARB-TC-06	0	0	0	Không master nào

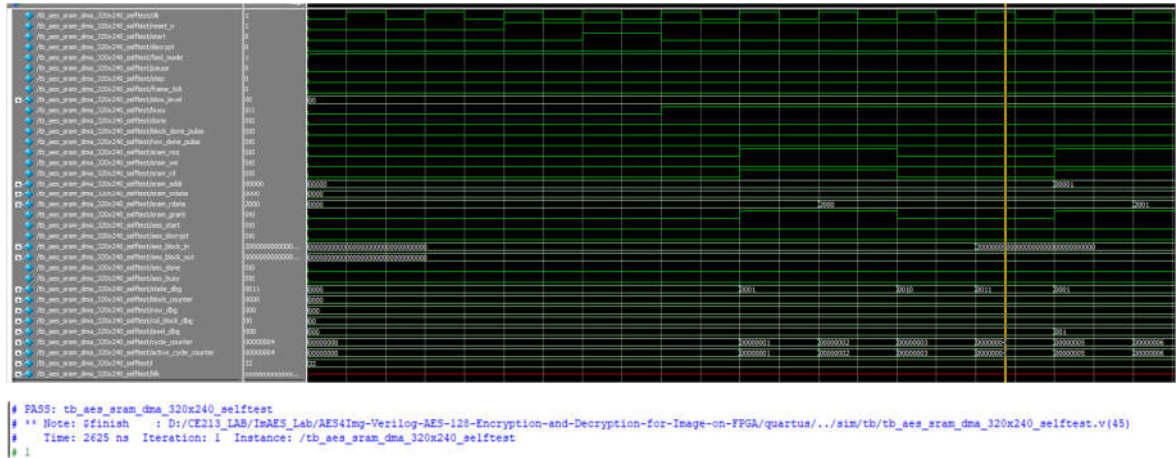
4.6 Kiểm thử AES SRAM DMA

AES SRAM DMA là module có độ phức tạp cao vì kết hợp nhiều thao tác tuần tự. Testbench testbench kiểm thử DMA xử lý ảnh qua AES cần kiểm tra quá trình đọc 8 word từ SRAM, ghép thành block 128 bit, chờ AES, nhận block dữ liệu ra và ghi lại 8 word vào vùng đích. Để mô phỏng nhanh, testbench có thể dùng AES mock core thay vì lõi AES thật.

Một điểm cần kiểm tra kỹ là thứ tự ghép và tách pixel. Nếu DMA đọc đúng pixel nhưng ghép sai thứ tự byte hoặc word, kết quả AES có thể vẫn tạo dữ liệu ra nhưng khi giải mã sẽ không khôi phục đúng ảnh. Vì vậy, waveform nên thể hiện chỉ số pixel, bộ đếm block, địa chỉ SRAM, dữ liệu đọc từ SRAM, block AES đầu vào, block AES đầu ra và dữ liệu ghi trở lại SRAM.

Ngoài chức năng cơ bản, DMA cần kiểm tra done cuối ảnh, xung hoàn tất block, xung hoàn tất dòng, pause, step và slow mode nếu các tín hiệu này được sử dụng trong demo.

Các tín hiệu này không trực tiếp ảnh hưởng đến thuật toán AES nhưng ảnh hưởng đến khả năng quan sát và điều khiển hệ thống trên phần cứng.



Hình 30. Waveform kiểm thử DMA đọc SRAM, xử lý AES và ghi kết quả

Bảng 27. Testcase kiểm thử AES SRAM DMA

Test ID	Mục tiêu	Điều kiện	Kết quả mong đợi
DMA-TC-01	Đọc đủ 8 pixel từ vùng nguồn.	start bật ở chế độ encrypt.	aes block dữ liệu vào chứa đúng 8 word đã đọc.
DMA-TC-02	Ghi đủ 8 pixel kết quả.	tín hiệu AES hoàn tất bật và block dữ liệu ra hợp lệ.	8 word được ghi vào vùng đích.
DMA-TC-03	Chuyển block tiếp theo.	Hoàn thành một block.	bộ đếm block tăng và địa chỉ nguồn/đích tăng đúng.
DMA-TC-04	Hoàn thành toàn ảnh.	Xử lý đủ số block.	done bật và busy tắt.
DMA-TC-05	Chế độ decrypt.	decrypt bằng 1.	Nguồn là ENC và đích là DEC.

4.7 Kiểm thử VGA timing và hiển thị

Khối VGA được kiểm thử ở hai mức. Mức thứ nhất là timing, bao gồm tọa độ x, tọa độ y, hsync, vsync, vùng hiển thị hợp lệ và xung khung hình. Mức thứ hai là hiển thị dữ liệu, bao gồm chọn quadrant, đọc pixel từ SRAM và chuyển RGB565 sang tín hiệu màu VGA 10 bit. Testbench testbench kiểm thử timing VGA phục vụ kiểm tra timing cơ bản.

Khi kiểm thử VGA, không nhất thiết phải mô phỏng toàn bộ nhiều frame vì thời gian mô phỏng có thể dài. Có thể tập trung kiểm tra các thời điểm biên như bắt đầu dòng, kết thúc vùng visible, vùng blanking, bắt đầu frame và xung khung hình. Với renderer, cần

kiểm tra tọa độ thuộc từng quadrant để bảo đảm địa chỉ SRAM được chọn đúng vùng ORIG, ENC hoặc DEC.

Kết quả phần cứng nên được thể hiện bằng ảnh chụp màn hình VGA. Trong bản báo cáo hiện tại, các vị trí này được để placeholder. Khi hoàn thiện, người thực hiện nên thay bằng ảnh minh họa trạng thái nạp ảnh, ảnh minh họa trạng thái mã hóa, ảnh minh họa trạng thái giải mã hoặc ảnh chụp trực tiếp từ màn hình thực tế.

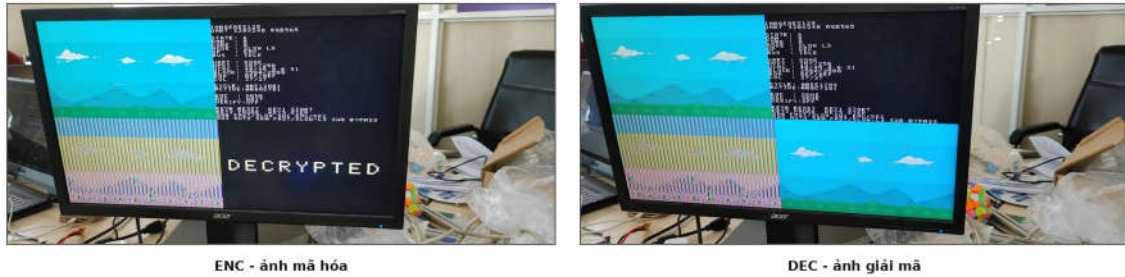


Hình 31. Waveform kiểm thử VGA timing 640x480



Hình 32. Ảnh chụp VGA sau khi nạp ảnh gốc

Ảnh VGA sau mã hóa và giải mã



Hình 33. Ảnh chụp VGA sau khi mã hóa và giải mã

4.8 Kiểm thử tích hợp toàn hệ thống

Kiểm thử tích hợp nhằm xác nhận các module phối hợp đúng theo luồng vận hành tổng thể. Testbench testbench kiểm thử tích hợp rút gọn có thể sử dụng kích thước ảnh rút gọn để giảm thời gian mô phỏng. Mục tiêu của smoke test không phải là kiểm tra toàn bộ 320x240 mà là xác nhận controller, loader, DMA, AES wrapper và VGA có thể hoạt động cùng nhau mà không bị treo trạng thái.

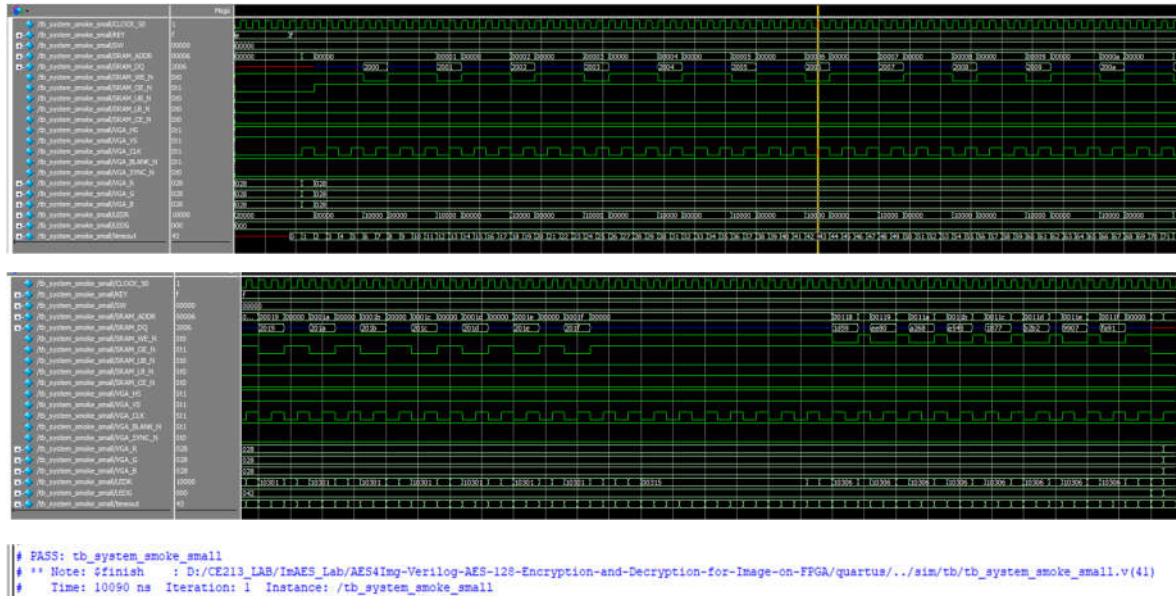
Kịch bản kiểm thử tích hợp bắt đầu bằng reset, sau đó nạp dữ liệu ảnh hoặc dùng bypass, kích hoạt start, chờ DMA chạy mã hóa, xác nhận cờ mã hóa hoàn tất và nếu auto mode được bật thì tiếp tục chờ giải mã. Trong waveform, cần quan sát trạng thái điều khiển, bắt đầu DMA, DMA đang bận, DMA hoàn tất, tín hiệu bắt đầu AES, tín hiệu AES hoàn tất, chủ truy cập bus và các cờ mã hóa hoàn tất hoặc cờ giải mã hoàn tất.

Đối với phần cứng thật, kịch bản kiểm thử gồm nạp cấu hình SOF, kết nối VGA và UART, gửi ảnh bằng tool Python, chờ dashboard báo ảnh đã nạp, nhấn start và quan sát kết quả. Nếu hệ thống hoạt động đúng, vùng ORIG hiển thị ảnh gốc, vùng ENC hiển thị dữ liệu mã hóa dạng khó nhận dạng và vùng DEC khôi phục lại ảnh gần với ảnh gốc.

Bảng 28. Kịch bản kiểm thử tích hợp hệ thống

Bước	Thao tác	Kết quả mong đợi
1	Reset hệ thống bằng KEY[0].	Controller về trạng thái khởi tạo, SRAM clear nếu được bật.
2	Gửi ảnh từ PC bằng tool Python.	số packet đã nhận tăng đến 600 và cờ ảnh đã nạp bật.
3	Nhấn KEY[1] để start.	DMA bắt đầu chạy theo chế độ được chọn.
4	Chạy encrypt.	Vùng ENC được ghi và cờ mã hóa hoàn tất bật.

Bước	Thao tác	Kết quả mong đợi
5	Chạy decrypt hoặc auto decrypt.	Vùng DEC được ghi và cờ giải mã hoàn tất bật.
6	Quan sát VGA.	ORIG, ENC, DEC và dashboard hiển thị đúng.



Hình 34. Waveform smoke test tích hợp hệ thống

4.9 Kết quả tổng hợp trên Quartus

Kết quả tổng hợp là căn cứ để đánh giá thiết kế có thể triển khai trên FPGA mục tiêu hay không. Trong phạm vi báo cáo, kết quả cần thể hiện rằng thiết kế đã đi qua các bước synthesis, fitter, assembler và timing analysis. Đây là cơ sở để chứng minh hệ thống không chỉ dừng ở mô phỏng mà còn có khả năng cấu hình lên FPGA.

Thiết kế sử dụng 13.761 logic elements trên tổng 33.216 logic elements, tương ứng khoảng 41 phần trăm. Số register sử dụng là 1.490, tương ứng khoảng 4 phần trăm. Tổng số pin sử dụng là 126 trên 475 pin, tương ứng khoảng 27 phần trăm. Bộ nhớ trong sử dụng 2.048 bit, nhỏ hơn 1 phần trăm, vì ảnh chính được lưu trong SRAM ngoài thay vì RAM nội bộ.

Kết quả timing cho thấy Fmax ở slow model là 77,42 MHz, cao hơn clock hệ thống 50 MHz. Slack setup cho CLOCK_50 là 7,084 ns và hold slack là 0,391 ns. Điều này cho thấy thiết kế có margin timing dương trong phạm vi báo cáo Quartus hiện tại. Khi viết báo cáo cuối cùng, nên chèn ảnh chụp summary của Quartus hoặc bảng trích xuất như dưới đây.

Bảng 29. Kết quả tài nguyên tổng hợp Quartus cho thiết kế chính

Chỉ tiêu	Giá trị	Tỷ lệ	Nhận xét
Total logic elements	13.761 / 33.216	41 %	Chiếm phần tài nguyên đáng kể nhưng vẫn còn dư.
Dedicated logic registers	1.490 / 33.216	4 %	Số thanh ghi không cao so với tổng tài nguyên.
Total pins	126 / 475	27 %	Phù hợp do hệ thống dùng SRAM, VGA, UART, KEY, SW và LED.
Total memory bits	2.048 / 483.840	< 1 %	Bộ nhớ trong ít vì ảnh nằm ở SRAM ngoài.
Embedded multipliers	0 / 70	0 %	AES và logic hệ thống không dùng DSP multiplier.
Total PLLs	0 / 4	0 %	Clock VGA 25 MHz được chia từ 50 MHz bằng logic.

Bảng 30. Kết quả timing từ TimeQuest

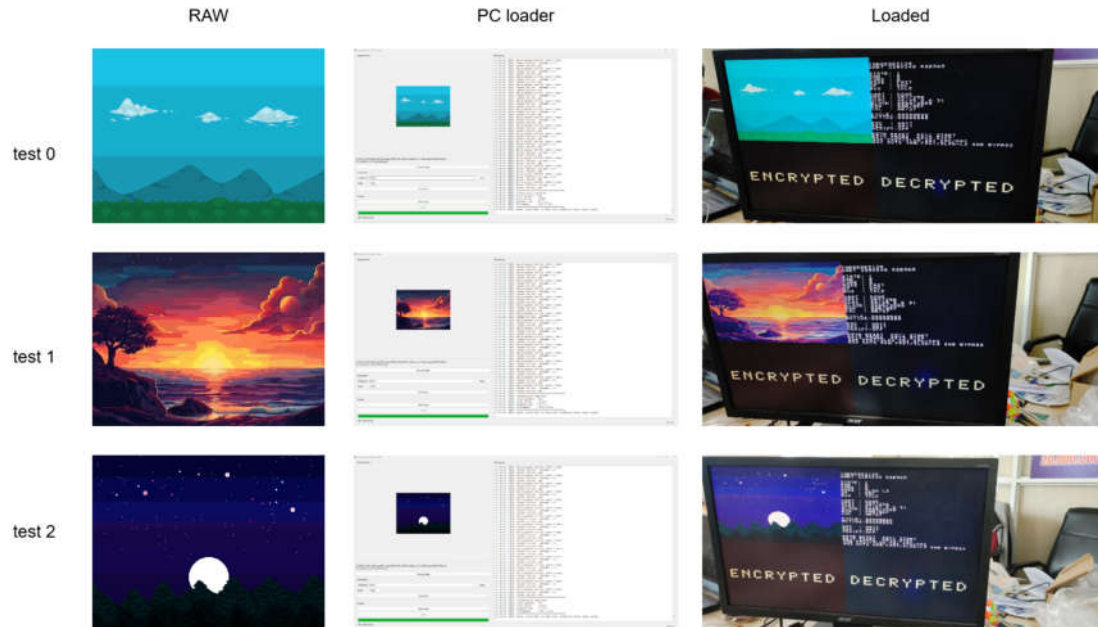
Thông số	Giá trị	Nhận xét
Clock constraint	20 ns/50 MHz	Clock hệ thống chính.
Slow Model Fmax	77,42 MHz	Cao hơn clock 50 MHz.
Slow Model Setup Slack	7,084 ns	Slack dương, đạt yêu cầu setup.
Slow Model Hold Slack	0,391 ns	Slack dương, đạt yêu cầu hold.
Minimum Pulse Width Slack	7,873 ns	Không vi phạm pulse width trong báo cáo.

4.10 Phân tích kết quả demo ảnh

Kết quả demo ảnh cần thể hiện được ba trạng thái chính. Trạng thái thứ nhất là ảnh đã nạp vào vùng ORIG. Trạng thái thứ hai là ảnh sau mã hóa trong vùng ENC. Ở trạng thái này, ảnh thường không còn nhận dạng trực quan do dữ liệu pixel đã bị biến đổi bởi AES. Trạng thái thứ ba là ảnh sau giải mã trong vùng DEC. Nếu hệ thống đúng, ảnh giải mã phải khôi phục lại nội dung gần với ảnh gốc.

Các ảnh kết quả minh họa cần được thay vào các placeholder bên dưới khi hoàn thiện báo cáo. Nên sử dụng ảnh chụp màn hình VGA ở ba trạng thái chính là nạp ảnh gốc, mã hóa và giải mã. Khi chèn ảnh, cần giữ caption rõ ràng và thêm đoạn nhận xét ngay sau hình để giải thích ý nghĩa của kết quả.

Nếu ảnh giải mã không khớp ảnh gốc, các nguyên nhân cần kiểm tra gồm byte order RGB565, thứ tự pack và unpack trong DMA, thứ tự block trong SRAM, vùng địa chỉ nguồn và đích, chế độ encrypt hoặc decrypt, khóa AES và việc dữ liệu ENC đã được tạo trước khi chạy decrypt hay chưa.



Hình 35. Kết quả các testcase sau khi nạp ảnh gốc

[illegible]

Hình 36. Kết quả ảnh sau khi mã hóa AES-128

test 0



test 1



test 2



Hình 37. Kết quả test0 sau khi giải mã AES-128

Bảng 31. Bảng tổng hợp đánh giá kết quả hệ thống

Tiêu chí	Kết quả quan sát	Nhận xét
Nạp ảnh UART	Tool gửi ảnh theo 600 packet, FPGA có cờ cờ ảnh đã nạp.	Đáp ứng mục tiêu nạp ảnh từ PC.

Tiêu chí	Kết quả quan sát	Nhận xét
Mã hóa AES	Vùng ENC hiển thị dữ liệu không còn nhận dạng trực quan.	Phù hợp với kỳ vọng của mã hóa ảnh theo block.
Giải mã AES	Vùng DEC khôi phục lại ảnh quan sát được.	Chứng minh luồng decrypt hoạt động trong phạm vi demo.
Hiển thị VGA	ORIG, ENC, DEC và dashboard cùng xuất hiện trên màn hình.	Hỗ trợ kiểm tra trực quan.
Tổng hợp Quartus	Flow successful, timing slack dương.	Thiết kế có thể triển khai trên FPGA mục tiêu.

4.11 Hạn chế trong phạm vi kiểm thử

Mặc dù hệ thống đã có nhiều testbench và kết quả tổng hợp, vẫn cần nhìn nhận một số hạn chế trong phạm vi kiểm thử. Thứ nhất, tốc độ UART 115200 baud làm thời gian nạp ảnh tương đối dài. Điều này phù hợp với demo nhưng chưa tối ưu nếu cần truyền ảnh liên tục hoặc ảnh có kích thước lớn hơn.

Thứ hai, cờ cờ kiểm chứng đạt hiện ở mức marker hệ thống, chưa phải là một DMA compare pass đầy đủ đọc toàn bộ ORIG và DEC để so sánh từng pixel trên phần cứng. Vì vậy, việc kết luận ảnh giải mã đúng hoàn toàn vẫn cần thêm cơ chế verify tự động hoặc readback về PC để so sánh dữ liệu.

Thứ ba, thiết kế hiện tại xử lý ảnh kích thước cố định 320x240 và packet có payload cố định 256 byte. Đây là cách đơn giản và ổn định cho demo, nhưng chưa hỗ trợ ảnh có kích thước động. Nếu mở rộng, packet nên có thêm metadata về width, height, format, mode và checksum toàn ảnh.

Thứ tư, hệ thống dùng AES theo từng block độc lập nên chưa bàn đến chế độ mã hóa khối như CBC hoặc CTR. Với mục tiêu học thuật và demo phần cứng, cách này giúp thiết kế dễ hiểu. Tuy nhiên, nếu hướng đến ứng dụng bảo mật thực tế, cần bổ sung chế độ mã hóa phù hợp và cơ chế quản lý IV hoặc nonce.

Bảng 32. Hạn chế và hướng xử lý đề xuất

Hạn chế	Ảnh hưởng	Hướng xử lý
UART 115200 baud còn chậm.	Thời gian nạp ảnh dài.	Tăng baudrate hoặc dùng giao tiếp nhanh hơn.
Chưa có DMA compare pass đầy đủ.	Chưa tự động xác nhận DEC bằng ORIG trên phần cứng.	Bổ sung module so sánh SRAM ORIG và DEC.
Ảnh cố định 320x240.	Chưa linh hoạt với nhiều kích thước ảnh.	Thêm header metadata trong packet.

Hạn chế	Ảnh hưởng	Hướng xử lý
Chưa dùng chế độ mã hóa khối nâng cao.	Chưa phù hợp cho ứng dụng bảo mật hoàn chỉnh.	Nghiên cứu CTR hoặc CBC trong phiên bản sau.

CHƯƠNG 5. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

5.1 Mức độ hoàn thành mục tiêu

Đề tài đã xây dựng được một kiến trúc hệ thống mã hóa và giải mã ảnh RGB565 bằng AES-128 trên FPGA. Hệ thống có luồng dữ liệu hoàn chỉnh từ PC đến FPGA qua UART, từ UART vào SRAM, từ SRAM qua DMA đến AES core và từ SRAM ra VGA. Đây là kết quả quan trọng vì đề tài không chỉ kiểm thử thuật toán AES ở mức đơn lẻ mà còn tích hợp thuật toán vào một hệ thống xử lý ảnh có ngoại vi đầy đủ.

Thiết kế hệ thống đề xuất đã được tổ chức lại theo hướng rõ ràng hơn, trong đó khối kết nối phần cứng mức cao là khối kết nối phần cứng chính và khối mô phỏng mức cao là khối mô phỏng và kiểm tra rút gọn. Các nhóm mã nguồn RTL, mô phỏng, công cụ hỗ trợ, tài liệu và cấu hình tổng hợp được tách biệt với nhóm thành phần kỹ thuật. Cách tổ chức này giúp người khác có thể đọc thiết kế, chạy mô phỏng, tổng hợp project và thay thế ảnh demo dễ dàng hơn.

Các testbench bao phủ nhiều khối quan trọng như giao diện lõi AES, phân xử SRAM, nạp ảnh, DMA xử lý ảnh, ghi packet và timing VGA. Điều này cho thấy hệ thống có cơ sở kiểm chứng tốt trong phạm vi đề tài, đồng thời giúp phát hiện lỗi tích hợp trước khi triển khai trên phần cứng.

Bảng 33. Đối chiếu mục tiêu và mức độ hoàn thành

Mục tiêu	Mức độ hoàn thành	Minh chứng
Nạp ảnh từ PC qua UART.	Hoàn thành ở mức thiết kế và có module/testbench.	Khối nạp ảnh và định dạng packet.
Lưu ảnh trong SRAM ngoài.	Hoàn thành ở mức kiến trúc và tổng hợp.	SRAM map ORIG, ENC, DEC và arbiter.
Mã hóa AES-128.	Hoàn thành.	AES wrapper và known-answer self-test.
Giải mã AES-128.	Hoàn thành.	AES decryption core và chế độ decrypt.
Hiển thị VGA.	Hoàn thành ở mức hệ thống.	Khối timing, đọc pixel và dựng khung hình VGA.
Kiểm thử mô phỏng và tổng hợp.	Hoàn thành trong phạm vi thiết kế.	Self-test, Quartus flow và timing summary.

5.2 Ưu điểm của hệ thống

Ưu điểm đầu tiên của hệ thống là kiến trúc phân cấp rõ ràng. Mỗi khối đảm nhiệm một chức năng cụ thể và giao tiếp với các khối khác bằng tín hiệu handshake hoặc bus rõ nghĩa.

Điều này giúp quá trình đọc thiết kế, mô phỏng và bảo trì thuận tiện hơn so với việc gộp nhiều chức năng vào một khối điều khiển lớn.

Ưu điểm thứ hai là hệ thống có khả năng demo trực quan. Việc hiển thị đồng thời ảnh gốc, ảnh mã hóa, ảnh giải mã và dashboard trên VGA giúp người dùng hiểu rõ hơn quá trình xử lý. Đây là điểm phù hợp với báo cáo đồ án vì kết quả không chỉ nằm ở log mô phỏng mà còn có thể quan sát trực tiếp bằng hình ảnh.

Ưu điểm thứ ba là hệ thống có cả công cụ phía máy tính và testbench phía FPGA. Công cụ phía máy tính hỗ trợ chuyển ảnh và gửi packet qua UART, trong khi testbench giúp kiểm tra từng khối chức năng. Cách kết hợp phần mềm và phần cứng này phản ánh đúng đặc trưng của một hệ thống nhúng hoặc hệ thống số hoàn chỉnh.

Ưu điểm thứ tư là thiết kế có kết quả tổng hợp và timing rõ ràng. Các chỉ số tài nguyên, Fmax và slack giúp báo cáo có cơ sở định lượng thay vì chỉ mô tả chức năng. Điều này làm tăng độ tin cậy của phần đánh giá kết quả.

5.3 Hạn chế còn tồn tại

Hạn chế đầu tiên là tốc độ nạp ảnh qua UART còn thấp. Với baudrate 115200, thời gian nạp ảnh 320x240 RGB565 không thể quá ngắn. Điều này không ảnh hưởng nhiều đến demo đơn ảnh nhưng sẽ hạn chế nếu muốn xử lý nhiều ảnh liên tục hoặc truyền video.

Hạn chế thứ hai là hệ thống chưa có cơ chế verify tự động toàn ảnh trên phần cứng. Việc quan sát ảnh DEC bằng mắt giúp kiểm tra trực quan nhưng chưa thay thế được so sánh dữ liệu từng pixel. Một DMA compare pass hoặc cơ chế readback về PC sẽ giúp đánh giá chính xác hơn.

Hạn chế thứ ba là thiết kế hiện tại cố định nhiều tham số như kích thước ảnh, payload packet và định dạng RGB565. Cách làm này giúp hệ thống ổn định và dễ kiểm thử, nhưng chưa linh hoạt nếu muốn mở rộng sang ảnh có kích thước khác.

Hạn chế thứ tư là hệ thống chưa triển khai các chế độ mã hóa khối phù hợp hơn cho bảo mật ảnh trong ứng dụng thực tế. Mỗi block được xử lý độc lập nên có thể còn lộ cấu trúc nếu dữ liệu ảnh có vùng lặp. Đây là điểm cần lưu ý nếu đề tài được mở rộng từ demo học thuật sang ứng dụng bảo mật thực tế.

5.4 Hướng phát triển

Trong tương lai, hệ thống có thể được cải tiến theo hướng tăng tốc giao tiếp giữa PC và FPGA. Một hướng đơn giản là tăng baudrate UART sau khi xác nhận adapter và timing ổn định. Một hướng mạnh hơn là sử dụng giao tiếp USB, Ethernet hoặc SD card để truyền ảnh nhanh hơn. Khi tốc độ truyền được cải thiện, hệ thống có thể xử lý ảnh lớn hơn hoặc nhiều ảnh liên tiếp.

Hướng phát triển tiếp theo là bổ sung cơ chế verify tự động. Module compare có thể đọc từng word từ vùng ORIG và DEC, so sánh dữ liệu và xuất số lượng pixel sai. Kết quả này có thể hiển thị lên dashboard hoặc gửi về PC. Khi có verify tự động, báo cáo có thể đưa thêm chỉ số định lượng thay vì chỉ dựa vào quan sát hình ảnh.

Hệ thống cũng có thể mở rộng packet UART để hỗ trợ metadata. Thay vì cố định 320x240, packet đầu tiên có thể chứa width, height, format, mode và checksum toàn ảnh. Nhờ đó, FPGA có thể xử lý nhiều cấu hình ảnh khác nhau. Tuy nhiên, việc này yêu cầu thiết kế lại bộ đếm địa chỉ, DMA và layout VGA.

Về thuật toán, hệ thống có thể bổ sung các chế độ mã hóa khối như CTR hoặc CBC để tăng tính thực tế. Với CTR, hệ thống cần tạo counter hoặc nonce cho từng block. Với CBC, hệ thống cần quản lý vector khởi tạo và phụ thuộc giữa các block. Hai hướng này đều làm tăng độ phức tạp nhưng giúp dễ tài gần hơn với ứng dụng bảo mật dữ liệu thực tế.

Về kiểm thử, hệ thống có thể bổ sung quy trình regression tự động để chạy toàn bộ testbench, thu log, sinh báo cáo pass fail và lưu waveform quan trọng. Báo cáo cũng có thể bổ sung biểu đồ thời gian xử lý, throughput theo mode và bảng so sánh tài nguyên giữa các phiên bản thiết kế.

Bảng 34. Các hướng phát triển đề xuất

Hướng phát triển	Lợi ích	Độ ưu tiên
Tăng baudrate hoặc thay giao tiếp nhanh hơn.	Giảm thời gian nạp ảnh.	Cao
Bổ sung DMA compare ORIG và DEC.	Xác nhận đúng dữ liệu tự động trên phần cứng.	Cao
Thêm metadata trong packet.	Hỗ trợ ảnh kích thước và định dạng linh hoạt hơn.	Trung bình
Bổ sung chế độ CTR hoặc CBC.	Tăng tính thực tế về bảo mật.	Trung bình
Tự động hóa regression test.	Tăng độ tin cậy khi refactor.	Cao
Tối ưu AES pipeline hoặc tài nguyên.	Cải thiện throughput hoặc giảm logic elements.	Trung bình

5.5 Kết luận chung

Tổng kết lại, đề tài đã thể hiện được quá trình thiết kế một hệ thống số hoàn chỉnh trên FPGA từ thuật toán đến ngoại vi. Hệ thống không chỉ là một lõi AES riêng lẻ mà là một kiến trúc xử lý ảnh gồm khối nạp ảnh qua UART, hệ thống SRAM, DMA, giao diện AES, hiển thị VGA và điều khiển trung tâm. Việc tổ chức này giúp người học hiểu rõ hơn cách một thuật toán được đưa vào phần cứng và tích hợp với các khối ngoại vi thực tế.

Thông qua đề tài, nhóm sinh viên củng cố được kiến thức về Verilog HDL, thiết kế FSM, xử lý dữ liệu song song, giao tiếp UART, tổ chức SRAM, hiển thị VGA và kiểm thử bằng testbench. Những nội dung này có tính nền tảng đối với các bài toán lớn hơn như xử lý ảnh thời gian thực, tăng tốc phần cứng, bảo mật dữ liệu nhúng và thiết kế hệ thống số trên FPGA.

Bản báo cáo hiện tại được xây dựng với các placeholder hình ảnh để thuận tiện cho việc hoàn thiện. Khi nộp chính thức, người thực hiện nên thay placeholder bằng sơ đồ kiến trúc, waveform, ảnh chụp VGA, ảnh chụp Quartus summary và kết quả chạy tool PC. Việc bổ sung hình thật sẽ làm báo cáo trực quan hơn và tăng sức thuyết phục cho phần đánh giá kết quả.

DANH MỤC TÀI LIỆU THAM KHẢO

- [1] National Institute of Standards and Technology, “Advanced Encryption Standard (AES),” FIPS PUB 197, Nov. 2001, updated May 2023. [Online]. Available: <https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.197-upd1.pdf>. Accessed: 25-May-2026.
- [2] National Institute of Standards and Technology, “NIST updates FIPS 197, Advanced Encryption Standard (AES),” May 2023. [Online]. Available: <https://csrc.nist.gov/news/2023/nist-updates-fips-197-advanced-encryption-standard>. Accessed: 25-May-2026.
- [3] Nguyễn Quân, “[AES] Bài 1 - Lý thuyết về mã hóa AES-128,” VLSI Technology, Sep. 2019. [Online]. Available: <https://nguyenquanicd.blogspot.com/2019/09/aes-bai-1-ly-thuyet-ve-ma-hoa-aes-128.html>. Accessed: 25-May-2026.
- [4] Nguyễn Quân, “[AES] Bài 2 - Lý thuyết về giải mã AES-128,” VLSI Technology, Oct. 2019. [Online]. Available: <https://nguyenquanicd.blogspot.com/2019/10/aes-bai-2-ly-thuyet-ve-giai-ma-aes-128.html>. Accessed: 25-May-2026.
- [5] IEEE Computer Society, “IEEE Standard for Verilog Hardware Description Language,” IEEE Std 1364-2005, Apr. 2006.
- [6] Terasic Technologies, “DE2-70 Development and Education Board User Manual,” Version 1.05, 2008.
- [7] Z. Navabi, Verilog Digital System Design: RT Level Synthesis, Testbench, and Verification, 2nd ed. New York, NY, USA: McGraw-Hill, 2006.
- [8] hệ thống đề xuất project thiết kế sistor, “Verilog AES-128 Encryption and Decryption for Image on FPGA,” tài liệu thiết kế cục bộ, 2026.
- [9] Intel Altera, “Quartus II Handbook and TimeQuest Timing Analyzer documentation,” Intel FPGA documentation, accessed 25-May-2026.
- [10] M. D. Ciletti, Advanced Digital Design with the Verilog HDL, 2nd ed. Upper Saddle River, NJ, USA: Prentice Hall, 2010.
-